

Table of Content

Declaration of Authorship and AI Usage.....	ii
Abstract.....	iv
Acknowledgement	v
List of Figures.....	x
List of Tables	xiv
List of Abbreviations	xvi
1 Chapter 1: Introduction.....	1
1.1 Introduction	1
1.2 Problem Definition.....	1
1.3 Project Objectives	4
1.4 Project Scope.....	4
1.5 Contributions of This Study	5
1.6 Project Timeline	5
1.7 Document Organization	6
1.8 Conclusion.....	7
2 Chapter 2: Literature Review	8
2.1 Introduction	8
2.2 Background	9
2.2.1 YARA	9
2.2.2 Sigma	11
2.2.3 Snort.....	12
2.3 Related Work.....	15
2.3.1 Review of Relevant Work.....	15
2.3.2 Issues with the current tools	19

2.3.3	Relationship Between Relevant Work and Our Own Work	20
2.4	Summary	22
3	Chapter 3: System Analysis.....	23
3.1	Introduction	23
3.2	Analysis of Existing Systems.....	23
3.3	Requirements Elicitation	24
3.3.1	Functional Requirements	24
3.3.2	Non-Functional Requirements	27
3.3.3	User Requirements and Domain Requirements.....	34
3.4	Requirements Specification.....	36
3.4.1	Use Case Diagram.....	36
3.5	Developmental Methodology.....	45
3.5.1	Working Procedure	46
3.5.2	Conditions and Tools Required	47
3.5.3	Project Constraints	48
3.6	Data Collecting and Processing.....	49
3.7	Summary	50
4	Chapter 4: System Design.....	52
4.1	Introduction	52
4.2	Architectural Design	52
4.3	Object-Oriented Design.....	53
4.3.1	Structural Static Models.....	54
4.3.2	Dynamic Models.....	59
4.4	Data Modeling.....	67
4.5	User Interface Design.....	68

4.6	Proposed Solution	74
4.6.1	Server Discovery Algorithm	74
4.6.2	Rule Distribution Workflow	74
4.6.3	Ingestion and Normalization Pipeline.....	74
4.6.4	Alerting and Dashboard Logic.....	75
4.6.5	IoC Scoring (<i>Future Enhancement</i>).....	75
4.7	Summary	75
5	Chapter 5: System Implementation	77
5.1	Introduction	77
5.2	Tools and Languages.....	77
5.3	Mapping Design to Implementation.....	77
5.4	Main/Most Important Codes	77
5.4.1	Model Building and Training.....	77
5.4.2	Model Testing (Experimental results)	77
5.4.3	System Development	77
5.5	System Testing	78
5.6	Results and Discussion.....	78
5.7	Summary	78
6	Chapter 6: Conclusion and Future Work	79
6.1	Conclusion.....	79
6.2	Goals Achieved	79
6.3	Limitations and Future Work	79
7	References	خطأ! الإشارة المرجعية غير معروفة.
8	Appendix A	88
9	Appendix B.....	89

1	Appendix C	91
----------	-------------------------	-----------

List of Figures

Figure 1: Current fragmented workflow, where YARA, Snort and LOKI run as separate scanners with manual rule updates and no centralized IoC management, leading to duplicated effort and visibility gaps.	2
Figure 2: Proposed Workflow, where our system manages and centralizes the various IoC scanner rules distribution. The different IoC scanners/rulesets (YARA, Snort, and LOKI) feed their detections into a single results view, providing centralized and correlated IoCs.....	2
Figure 3: The IoC “Pyramid of Pain,” illustrating how lower-level indicators are relatively easy for attackers to change, while higher-level indicators are harder to evade and therefore more painful for adversaries or attackers when defenders detect or block them.	3
Figure 4: Project schedule Gantt chart from September to December 2025.	6
Figure 5: Overall structure of the IoC Manager report, showing the main chapters and subsections.	7
Figure 6: YARA workflow, where a suspicious pattern from a malware is encoded as a YARA rule and then used by scanning tools to detect similar malware across other files and environments [13].	10
Figure 7: YARA rule-based detection workflow. [13].	10
Figure 8: Sigma rules where generic YAML-based detection rules are bundled into rule packs and passed through a SIEM-specific queries for platforms such as Splunk, Elastic, and other security tools.	11
Figure 9: Structure of a Snort rule, showing the rule header (action, protocol, source address/port, direction, destination address/port) and the rule options that describe the alert message and additional rule parameters.	13
Figure 10: Mapping Snort alert fields to corresponding IoC attributes.	14
Figure 11: High-level Snort-to-IoC Manager pipeline.	14
Figure 12: MISP website main page.	15
Figure 13: MISP threat-intelligence dashboard showing malicious activity events, visual relationship graphs, authentication-failure statistics, and gamified achievement indicators used to monitor and analyze security incidents.....	16
Figure 14: OpenCTI website main page.	17

Figure 15: OpenCTI threat-intelligence dashboard displaying high-level incident metrics, geographic distribution of activity, campaign statistics, and visual analytics (bar charts, heatmaps, and donut charts) to help analysts monitor and prioritize emerging threat.	17
Figure 16: FortiAnalyzer website main page.....	18
Figure 17: Fortinet FortiAnalyzer system and license information, resource usage, device status, and real-time log ingestion rates.....	18
Figure 18: “Faultlessness” reliability model, linking the high-level goal of “Faultlessness” to its qualities and the concrete tasks and mechanisms that make or help achieve each quality.	30
Figure 19: “Recoverability” reliability model, linking the high-level goal of “Recoverability” to its qualities and the concrete tasks and mechanisms that make or help achieve each quality.	30
Figure 20: “Recoverability” reliability model illustrating failure scenarios (e.g., Verify backup integrity) and how they impact the recoverability quality attribute.....	31
Figure 21: “Fault Tolerance” reliability model, linking the high-level goal of “Fault tolerance” to its qualities and the concrete tasks and mechanisms that make or help achieve each quality.	31
Figure 22: “Fault Tolerance” reliability model illustrating failure scenarios (e.g., failed/queued jobs) and how they impact the fault tolerance quality attribute.....	32
Figure 23: “Integrity” security model, linking the high-level goal of “Integrity” to its qualities and the concrete tasks and mechanisms that make or help achieve each quality.	32
Figure 24: “Confidentiality” security model, linking the high-level goal of “Confidentiality” to its qualities and the concrete tasks and mechanisms that make or help achieve each quality.	33
Figure 25: “Performance–speed” model, linking the high-level goal of “Performance–speed” to its qualities and the concrete tasks and mechanisms that make or help achieve each quality.	33
Figure 26: “Performance–space” model, linking the high-level goal of “Performance–space” to its qualities and the concrete tasks and mechanisms that make or help achieve each quality.	34
Figure 27: Use-case diagram showing how different actors (IT, Administrator, Analyst) interact with the system to perform various actions on the system.	36

Figure 28: “Agile development methodology”.....	45
Figure 29: Kanban board illustrating the project’s agile task tracking.	47
Figure 30: Project timeline view showing scheduled tasks over the sprint.	47
Figure 31: Visual overview of the IoC Manager environment and project constraints.....	50
Figure 32: High-level architectural design of our IoC Manager.....	53
Figure 33: Class diagram of the IoC Manager, showing its main internal objects (Scanner, Sweeper, Scheduler, Network/Target, ScanPlan, IOCFile/IOC, and Report) and how they interact with each other.	54
Figure 34: Entity–relationship diagram showing core entities (Sweeper, Network, Target, Scanner, ScanJob, User, Alert, ScanResult, Report, IOCFile, and IOC).....	58
Figure 35: State machine diagram for the IoC Manager, showing how the system transitions between the different system states.....	59
Figure 36: State machine diagram for the target server, showing how the target transitions between the different states.....	59
Figure 37: Activity diagram for the server sweeping process.	60
Figure 38: Activity diagram for IoC scanner rules distribution.....	61
Figure 39: High-level IoC Manager threat-detection flow, illustrating how data is collected, processed and forwarded to the IoC Manager for centralized alerting, dashboards and analyst feedback.	62
Figure 40: A sequence diagram illustrates how users initially interact with the IoC Manager through the login and authentication process.....	63
Figure 41: A sequence diagram shows the main interactions carried out by the Admin role within the IoC Manager. It highlights administrative tasks such as creating new user accounts, modifying existing IOC records, and performing data governance operations.....	64
Figure 42: A sequence Diagram that focuses on the workflow of the IT role, which is responsible for setting up operational environments within the IoC Manager.	65
Figure 43: A sequence diagram that outlines the interaction flow for the Analyst role, who primarily works with searches, and reports.	66
Figure 44: Relational ER Diagram (Logical Data Model) showing the main tables with their attributes, primary keys and foreign-key relationships.....	67
Figure 45: a dashboard providing an overview.....	68

Figure 46: a specific overview of IoC activity.....	69
Figure 47: 48a subpage detailing YARA rules. You can import rules or add one specific rule.	70
Figure 49: an overview of Threat Feeds, while being able to add more.....	70
Figure 50: Servers subpage which includes current servers and ability to connect to them. You can discover more servers.	71
Figure 51: Distribution subpage.....	71
Figure 52: you can generate specific types of reports, with options for formats, time and filters.	72
Figure 53: a visualization of data.....	72
Figure 54: history of actions for who did what.....	73
Figure 55: user settings.	73

List of Tables

Table 1: a comparison table of key features across threat intelligence platforms	22
Table 2: a table that shows the Non-Functional requirements and their specifications.....	29
Table 3: “Login” use case description.	37
Table 4: “Scan” use case description for executing scan plans by the IT actor.....	37
Table 5: “ScanSnort” use case description for Snort-based network scanning.	38
Table 6: “ScanSigma” use case description for Sigma-based log analysis.	38
Table 7: “ScanYARA” use case description for YARA-based file.....	39
Table 8: “ReceiveAlert” use case description for alert notification and viewing by IT.	39
Table 9: “AddRule” use case description for creating new IoC rules.	40
Table 10: “AddPlan” use case description for defining and saving scan plans.	40
Table 11: “Sweep” use case description for network sweep operations.....	41
Table 12: “Report” use case description for generating structured scan and alert reports.	42
Table 13: “SetAccess” use case description for managing user roles and permissions.....	42
Table 14: “Maintain” use case description for performing system maintenance operations..	43
Table 15: “Audit” use case description for reviewing system audit logs.	43
Table 16: “Analyze” use case description for analyst review of indicators, matches, and alerts.	44
Table 17: “ViewReport” use case description for viewing generated reports.	44
Table 18: “Search” use case description for querying and filtering data.....	45

List of Abbreviations

IoC	Indicator of Compromise
ML	Machine Learning
AI	Artificial Intelligence
YARA	Yet Another Recursive Acronym
GeoIP	Geographical Internet Protocol
TTP	Tactics, Techniques and Procedures
API	Application Programming Interface
ICMP	Internet Control Message Protocol
SSH	Secure Shell
IP	Internet Protocol
URL	Uniform Resource Locator
CVE	Common Vulnerabilities and Exposures
IDS	Intrusion Detection System
IPS	Intrusion Prevention System
SIEM	Security Information and Event Management
ER	Entity Diagram
RBAC	Role-Based Access Control
RTO	Recovery Time Objective
RPO	Recovery Point Objective
UI	User Interface

MTTR	Mean Time To Recover
IT	Information Technology
DB	Database
SCP	Secure Copy Protocol
SSO	Single Sign-On
SID	Snort Rule Identifier
OSS	Open Source Software
TXT	Text File Document
JSON	JavaScript Object Notation
CSV	Comma Separated Values
PDF	Portable Document Format
REST	Representational State Transfer
EDR	Endpoint Detection and Response
OS	Operating System
DNS	Domain Name System
MAC	Media Access Control
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
SQL	Structured Query Language
SDK	Software Development Kit
CAPTCHA	Completely Automated Public Turing test to tell Computers and Humans Apart

1 Chapter 1: Introduction

1.1 Introduction

The IoC Manager project provides a central platform for managing Indicators of Compromise (IoCs) from different sources and executing multiple IoC scanners/rulesets (YARA, Snort, Sigma, and Loki [1]). IoCs can describe malicious activity at different layers, such as network-based indicators and host-based indicators. In many environments, analysts must log into several scanner terminals and individual servers to upload IoC rule files, trigger scans, and manually collect results, which are slow, error-prone, and exhausting at scale. The IoC Manager addresses this by centrally ingesting IoCs, distributing IoC rule files to connected scanners and servers, and consolidating their detection results in one place so users such as analysts and administrators can quickly see where each IoC was matched and on which host.

The manager also includes a dashboard that gives users a unified overview of current IoC status and severity, connected servers, and recent detection matches, along with notifications for urgent alerts and completed jobs. The dashboard supports filtering (for example, by IoC source, status, severity, or server) and allows exporting the results as a structured report. By doing this, the program removes the fatigue of switching between multiple scanners and servers for viewing, distributing, and collecting IoC rules and results, and instead provides a single, consistent interface for managing IoCs and reviewing detection results.

1.2 Problem Definition

In many organizations, security teams use multiple IoC feeds and rely on several scanners and servers (for example Loki and Snort running on different machines). These feeds often contain both network-based indicators (such as malicious IP addresses, domains, and unusual traffic patterns) and host-based indicators (such as file hashes or unexpected processes on servers).

IoC rules must often be uploaded separately to each tool, scans triggered manually, and detection results collected from different terminals or dashboards. This fragmented workflow is time-consuming, error-prone, and makes it difficult to see where IoC rules are deployed, which hosts have been scanned, and where matches have occurred. There is no single, unified view for managing IoC rules and their detection results across scanners and servers.

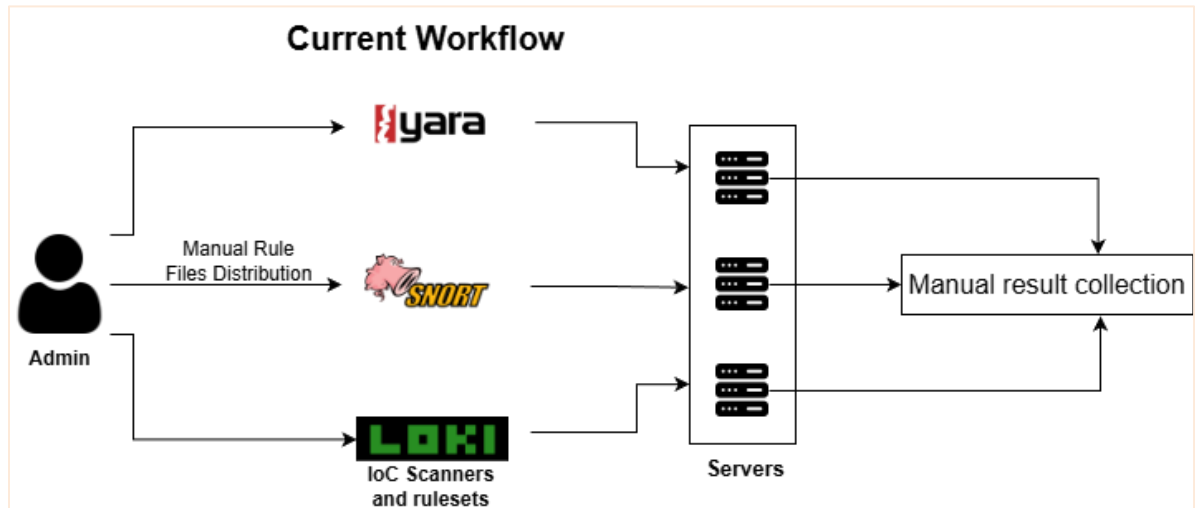


Figure 1: Current fragmented workflow, where YARA, Snort and LOKI run as separate scanners with manual rule updates and no centralized IoC management, leading to duplicated effort and visibility gaps.

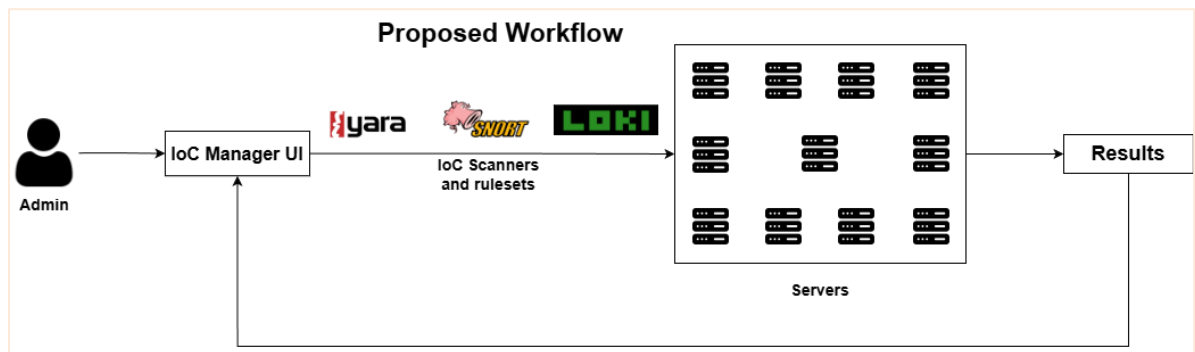


Figure 2: Proposed Workflow, where our system manages and centralizes the various IoC scanner rules distribution. The different IoC scanners/rulesets (YARA, Snort, and LOKI) feed their detections into a single results view, providing centralized and correlated IoCs.

IoCs can describe malicious activity at different layers, ranging from low-level technical values to higher-level behavioral patterns. The IoC “Pyramid of Pain” [2] groups these into hash values, IP addresses, domain names, host or network artifacts, tools, and finally tactics, techniques and procedures (TTPs). The lower layers (hashes, IPs, domains) are relatively easy for attackers to change, so blocking them often only causes trivial or simple disruption. In contrast, higher layers such as host and network artifacts, tools, and TTPs are harder for adversaries to modify without significantly altering their infrastructure or operations, which makes them more valuable but also more complex to track. Even at the lower and middle layers that this project focuses on (IP addresses and host/network artifacts) the volume and amount of indicators can be very high, increasing operational complexity for security teams and reinforcing the need for centralized IoC management.

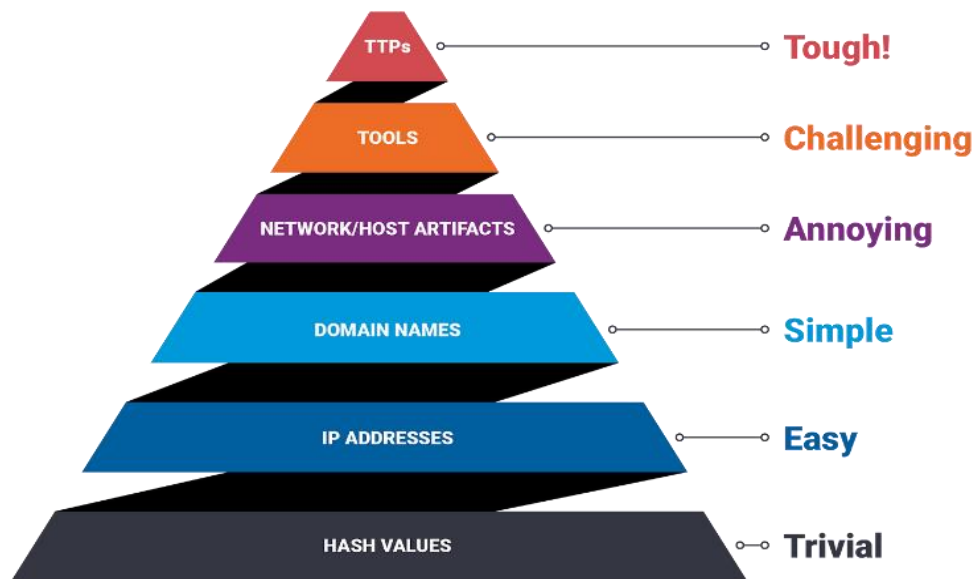


Figure 3: The IoC “Pyramid of Pain,” illustrating how lower-level indicators are relatively easy for attackers to change, while higher-level indicators are harder to evade and therefore more painful for adversaries or attackers when defenders detect or block them.

1.3 Project Objectives

To review existing Indicators of Compromise (IoC) management tools and network/server discovery approaches to understand their architecture, functionality, and limitations, with particular attention to how they handle network-based IoCs (malicious IP addresses, domains, unusual traffic) and server-side host-based IoCs (file hashes and unusual processes).

- To design and implement an IoC Manager backend that can centrally ingest and store IoCs, and distribute IoC rule files (YARA, Sigma, and Snort) to connected scanners and servers.
- To implement mechanisms that assist in discovering and managing servers that run IoC scanners, and for collecting their detection results into a single, consolidated view.
- To develop a web-based dashboard that shows IoC status (for network-based and server-side host-based indicators), affected servers, connected servers, and recent matches, with basic filtering and reporting, and to evaluate the project in a lab environment for correctness and reduction of manual effort.

1.4 Project Scope

In our project, the scope is to build an IoC Manager that centrally ingests and stores network and server-side host-based IoCs, distributes their corresponding YARA/Snort/Loki rule files to connected servers, discovers and tracks servers running these scanners, and collects their detection results into a unified dashboard for review. The system is designed to help cybersecurity analysts organize, validate, and search through these indicators efficiently within a centralized platform. The platform does not perform active malware analysis or automated threat response; its purpose is to provide structured storage, tagging, and basic enrichment (such as WHOIS and GeoIP) [3] [4] to support threat intelligence operations. The system operates in a controlled environment and serves as proof of concept for secure IOC management rather than a full-scale enterprise security solution.

1.5 Contributions of This Study

The Kingdom of Saudi Arabia has become one of the global leaders in cybersecurity, ranking first worldwide in the cybersecurity indicator of the IMD World Competitiveness Yearbook 2024 and 2025 [5], and being classified as a Tier-1 “role-modelling” nation in the ITU Global Cybersecurity Index 2024 [6]. National strategies and the National Cybersecurity Authority emphasize building local human capabilities and developing a domestic cybersecurity industry, rather than relying solely on imported solutions. Within this context, there is a growing need for practical, Saudi-developed tools that can be deployed in local environments and integrated with existing security operations. This study contributes to that ecosystem by designing and implementing an IoC Manager focused on network-based and server-side host-based indicators. This provides a concrete reference implementation of an IoC-orchestration layer tailored for multi-scanner environments rather than a single vendor product. The project offers an implementation approach and evaluation that can be reused or extended by future researchers and practitioners in Saudi Arabia. The design decisions, server-discovery mechanisms, and integration patterns with various IoC scanners can serve as a starting point for more advanced, production-grade platforms or for further academic work on IoC management and automation in the local cybersecurity community.

1.6 Project Timeline

This section summarizes the planned schedule for the early phases of the IoC Manager project. The main activities include problem definition, domain study, environment setup, and the initial analysis and engineering work.



Figure 4: Project schedule Gantt chart from September to December 2025.

1.7 Document Organization

This report is organized into six chapters to present the IoC Manager project in a logical, step-by-step manner. Chapter 1 introduces the topic, motivation, and objectives. Chapter 2 reviews the relevant literature and related work. Chapter 3 analyzes the system and derives the key requirements through requirements elicitation. Chapter 4 presents the system design, including the architectural design, object-oriented design, and the main structural/static models. Chapter 5 describes the implementation, highlighting the main/important code components and any model building/training activities. Finally, Chapter 6 concludes the report by summarizing the outcomes and outlining future improvements.

The following diagram illustrates the overall structure of the document:

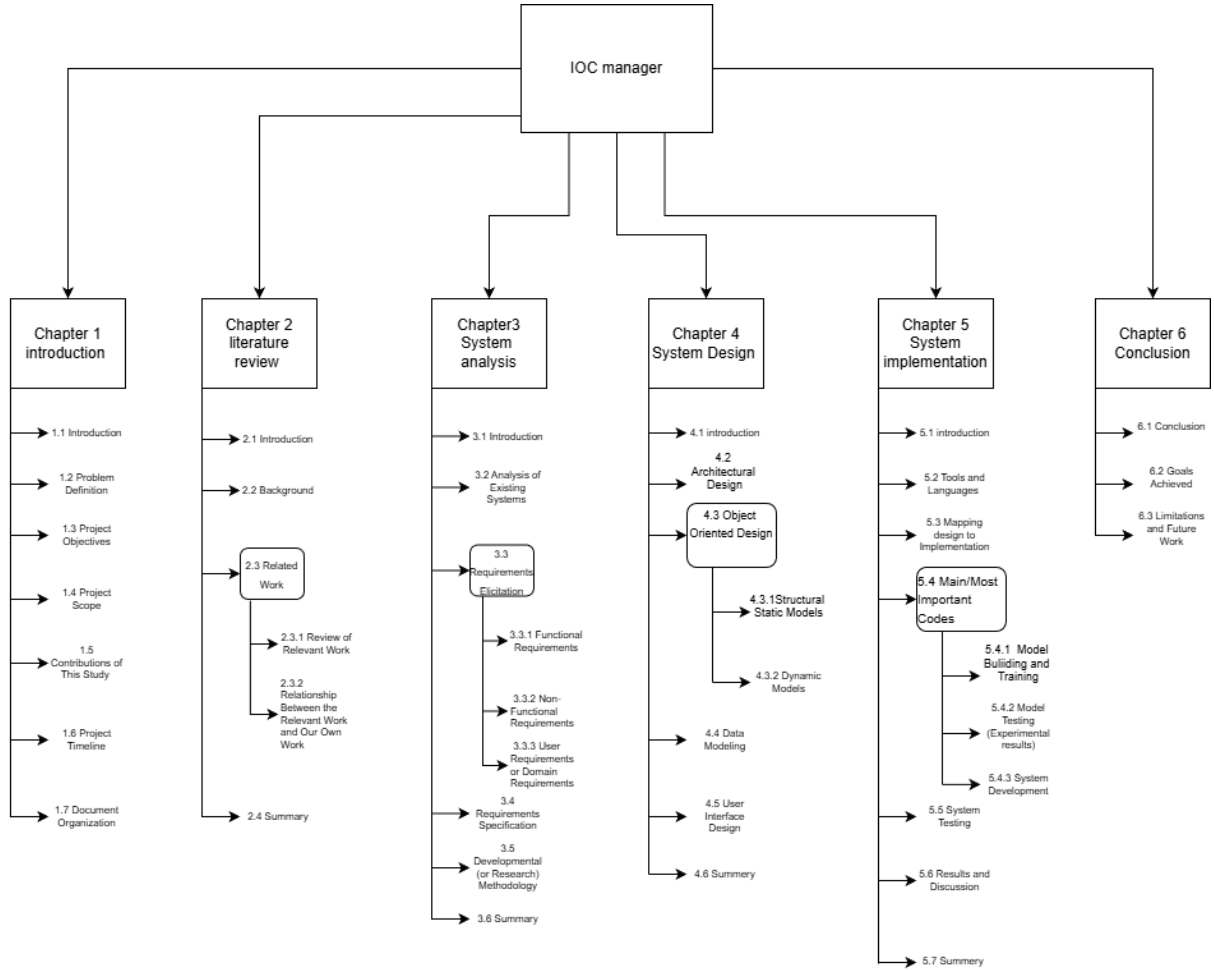


Figure 5: Overall structure of the IoC Manager report, showing the main chapters and subsections.

1.8 Conclusion

In this chapter, we introduced the overall context of the project by outlining the background and motivation behind the study. We defined the problem that the project aims to address and clarified the main objectives guiding the proposed work. The scope of the project was also outlined to specify the boundaries and limitations within which the study will be conducted. Additionally, the expected contributions of this work were briefly highlighted, along with a general project timeline that presents the major phases of the project.

Having established the foundation of the study, the next chapter presents a comprehensive literature review. It will examine previous research, existing methodologies, relevant theories, and current technological solutions related to the topic.

2 Chapter 2: Literature Review

2.1 Introduction

In the domain of cybersecurity operations, Indicators of Compromise are crucial for detecting threats across networks. As we mentioned in the first chapter, organizations rely on multiple rulesets, with them being distributed across environments. Despite their importance, the workflow is fragmented. Meaning, managing IoCs and distribution of threat detection files are all areas prone to error whilst being a burden for security teams. The purpose of this review will be to examine these existing IoC management tools, scanner integration methods, and centralized security platforms to highlight their current limitations.

Our project aims to address these gaps by integrating IoC scanners, automatic rule distribution, and result aggregation into one central platform. Unlike existing solutions, which focus on intelligence enhancement or post-incident response, the IoC Manager explores a unified model that seamlessly connects IoC rulesets (YARA, Snort, and Sigma) and target servers. This approach builds upon works in cyber threat intelligence, leveraging frameworks like the Pyramid of Pain to prioritize efficient detection across layers of indicators.

By centralizing management, distribution and consolidation of IoC rule files, our project removes the fatigue of switching between scanner terminals and servers for viewing, distributing, and collecting IoC rules and results. This approach not only fills the existing gaps in fragmented security workflows but also expands the scope of centralized management beyond simple data sharing, offering richer and more efficient operational experience for analysts.

This chapter will examine platforms in detail to identify their operational limits and to derive design requirements for a scanner-orchestration layer. By comparing intelligence management features, we generate the functional and non-functional requirements that motivate the IoC Manager.

2.2 Background

In this section, we will provide a foundational understanding of the IoC scanners and rulesets that will be utilized in the development of our project. From this point onwards, the document will get far more technically descriptive, but we will aim to present an overview regardless of experience level to equip readers with the necessary context to follow through.

2.2.1 YARA

YARA is an open-source tool widely used in malware research, threat intelligence, and digital forensics [7]. It allows analysts to create descriptive rules that characterize malware families or suspicious files based on identifiable patterns. These rules typically contain strings, hexadecimal sequences, or regular expressions that reflect known malware traits. It can scan files, directories, and memory dumps. It is considered an industry standard and is used by organizations, security vendors, and researchers for malware detection and hunting. [8] [9] [10]

YARA rules are written in a structured format composed of three primary sections:

- meta: contains metadata such as rule description, author, date, or reference.
- strings: defines text patterns, byte sequences, or regular expressions that YARA should search for.
- condition: a Boolean expression describing when a rule should trigger, usually combining one or more string matches. [11]

When scanning, YARA loads the rule files and matches their defined conditions against the content of the target files or memory. If one or more conditions are evaluated to be true, YARA reports a match along with the associated rule name. Unlike hash-based detection, which fails when malware is slightly modified, YARA can detect related variants because its rules are based on recurring characteristics rather than exact values [12].



Figure 6: YARA workflow, where a suspicious pattern from a malware is encoded as a YARA rule and then used by scanning tools to detect similar malware across other files and environments [13].

YARA is particularly effective for malware classification, threat hunting, and identifying suspicious artifacts. It is commonly integrated into automated analysis pipelines, sandbox systems, and endpoint detection tools. Its rule format is human-readable and easily maintained, allowing analysts to update signatures as new threats emerge [14] [15] [16].

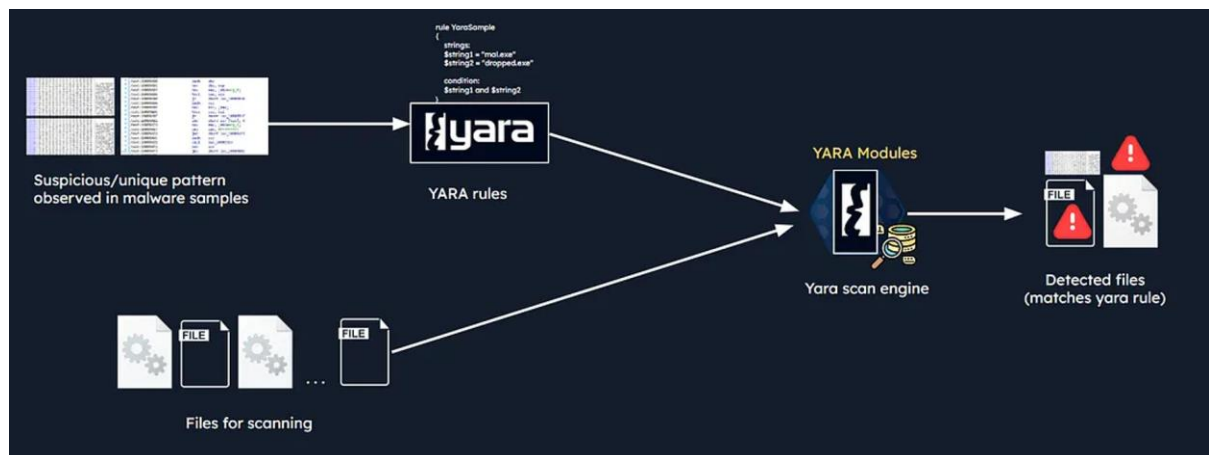


Figure 7: YARA rule-based detection workflow. [13].

The workflow, in order:

Collect IoCs → Create or Select YARA Rules → Distribute Rule Files → Perform Scanning → Collect Results → Analyze Matches → Update Rules

2.2.2 Sigma

Sigma is an open-source, vendor-neutral rule format designed to express detection logic for log data using a standardized YAML-based structure [17] [18]. As outlined in Chapter 1, Sigma functions as a generic signature language for log-based Indicators of Compromise (IoCs) across servers and infrastructure systems. Instead of writing different queries for each SIEM platform, security analysts write a single Sigma rule, which can then be translated into the native query language of different SIEMs via conversion backends. This makes Sigma well-suited for environments where multiple SIEM or log-analysis tools are in use [19].

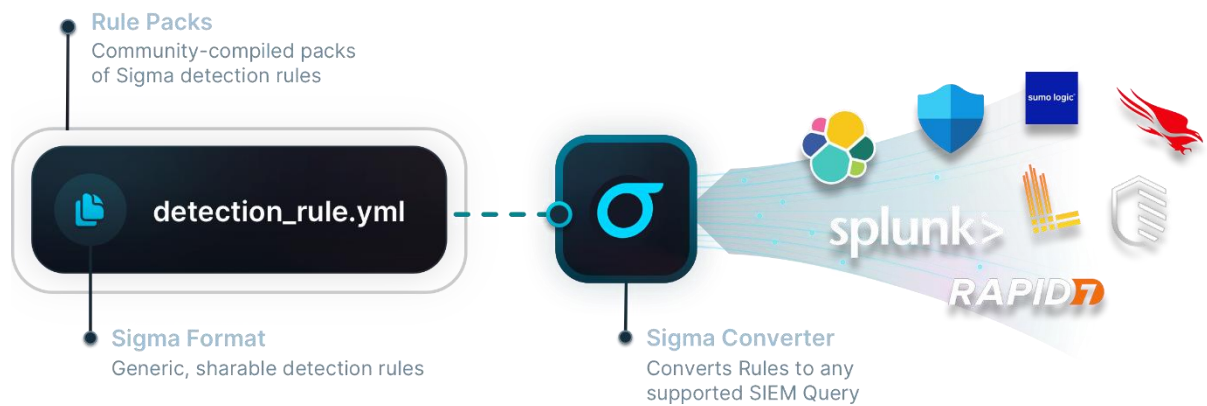


Figure 8: Sigma rules where generic YAML-based detection rules are bundled into rule packs and passed through a SIEM-specific queries for platforms such as Splunk, Elastic, and other security tools.

A Sigma rule consists of human-readable metadata (such as title, description, and tags), log source definitions, detection conditions, optional output fields, and a severity level. Analysts can describe log-based indicators or behavioral patterns using a unified syntax, which enhances readability and maintainability when writing and managing detection logic [20] [16].

A high-level example of the components is given below:

Section	Description
title	Human-readable name describing the detection rule
logsource	Specifies the type of logs and source product
detection	Defines conditions, keywords, or patterns to match within log entries
fields	Fields to include in the output
tags	MITRE ATT&CK mappings or classification metadata
level	Severity or confidence level

Table 1: Key components of a Sigma detection rule, summarizing the main sections and their corresponding descriptions.

Within the IoC Manager system, Sigma is one of the supported rule formats alongside YARA and Snort. Sigma rules stored in the IoC Manager represent log-based indicators or suspicious activity patterns derived from imported IoCs. When such rules are distributed to servers running log-processing components, their detection outputs are collected and aggregated by the IoC Manager. This ensures that log-based detections remain synchronized with the centralized IoC repository and provides analysts with a consistent view of threats across file-, network-, and log-based detection layers.

By incorporating Sigma into its rule ecosystem, the IoC Manager enables organizations to maintain a vendor-independent, unified log-detection capability that can adapt to heterogeneous logging infrastructures while leveraging a common indicator repository.

2.2.3 Snort

Snort is a free and open-source network intrusion detection and prevention system (IDS/IPS). Recognized as one of the most influential open-source tools [21]. Snort's popularity arises from its ability to perform real-time network traffic analysis and packet logging. [22] It can detect a wide range of network attacks, including port scans, OS fingerprinting attempts, and web-based exploits such as SQL injections [23]. By leveraging a large collection of community-maintained rules and signatures, Snort can identify known threats and suspicious network behaviors with high precision. [24] [25] [26]

Snort operates in three primary modes:

1. **Sniffing Mode** - Displays network packets in real time.
2. **Packet Logger Mode** - Logs captured packets to disk for later analysis.
3. **Network Intrusion Detection/Prevention Mode** - Analyzes live network traffic against predefined rules looking for potential intrusions.

Rules, which is the heart of Snort [27], consists of a header and options, combined to form a detection condition. For example [28]:

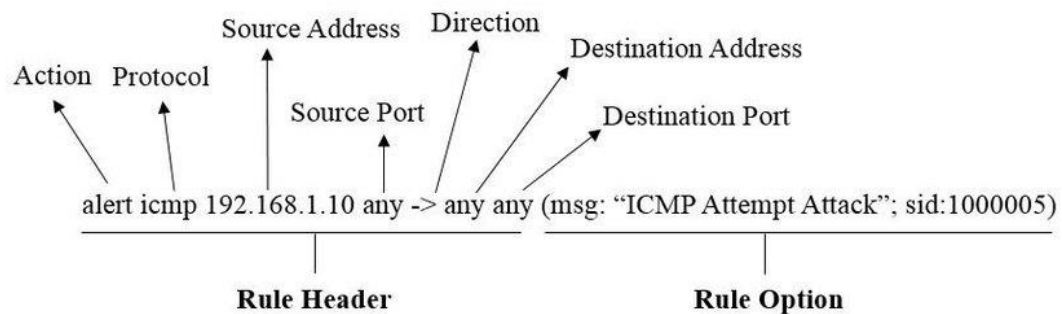


Figure 9: Structure of a Snort rule, showing the rule header (action, protocol, source address/port, direction, destination address/port) and the rule options that describe the alert message and additional rule parameters.

This rule triggers when an ICMP ping is detected and logs it with a descriptive message and a unique rule ID.

In our IoC Manager project, Snort is treated as one of the supported IoC scanner engines. Snort generates alerts based on rule files distributed by the IoC Manager, and these alerts are collected, normalized, and visualized in the dashboard so that raw network events are turned into structured, actionable detection results linked to specific IoCs and servers.

Each Snort alert field can be mapped to a corresponding Indicator of Compromise (IoC) attribute. For example:

Snort Field	IoC Field
Source IP	→ Attacker IP
Destination IP	→ Target System
Signature Message	→ Threat Type
Timestamp	→ Detection Time
Rule ID	→ Detection Rule Reference

Figure 10: Mapping Snort alert fields to corresponding IoC attributes.

Snort alerts generated on the network are treated as one of the main detection inputs to the IoC Manager. The system ingests these alerts, normalizes key fields such as source and destination IPs, timestamps, signature messages, and rule IDs, and stores them in the IoC database so they can be visualized through dashboards (for example, top attacker IPs, time-based attack trends, severity distributions, and event tables showing source, destination, and detection time). In summary, the integration can be viewed as a simple pipeline:

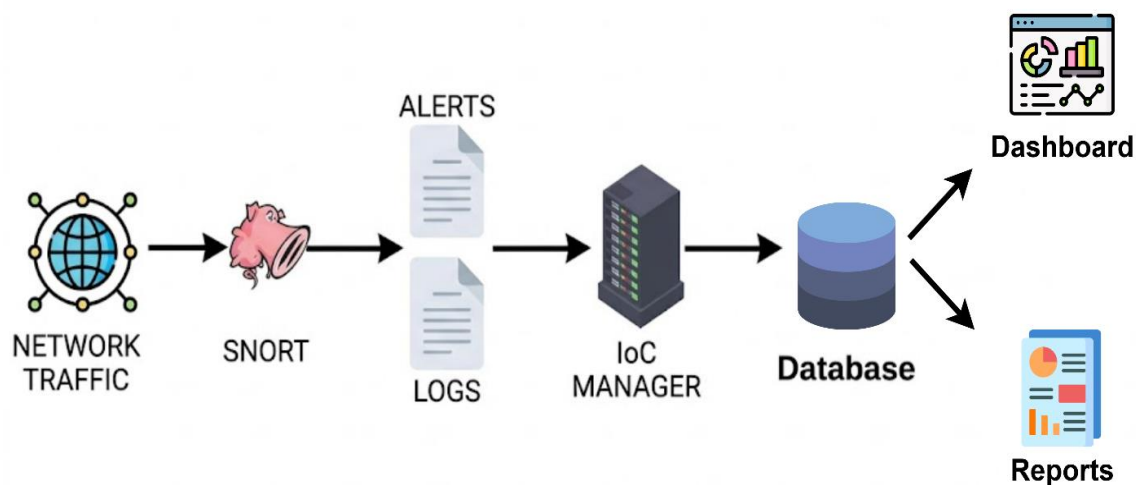


Figure 11: High-level Snort-to-IoC Manager pipeline.

2.3 Related Work

Existing IoC management and threat intelligence platforms attempt to improve the overall visibility, automate information sharing, and enhance intuition. However, as emphasized in Chapter 1, these platforms primarily focus on enriching threat data rather than orchestrating scanner operations across distributed servers.

This section reviews three major systems: MISP, OpenCTI, and Fortinet FortiAnalyzer to evaluate their strengths and identify gaps relevant to multi-scanner IoC rule distribution, unified detection, and result collection.

2.3.1 Review of Relevant Work

2.3.1.1 MISP (Malware Information Sharing Platform)

MISP is an open-source platform used for threat intelligence [29] and designed for collecting, storing, exchanging, and correlating Indicators of Compromise and broader threat intelligence [29].

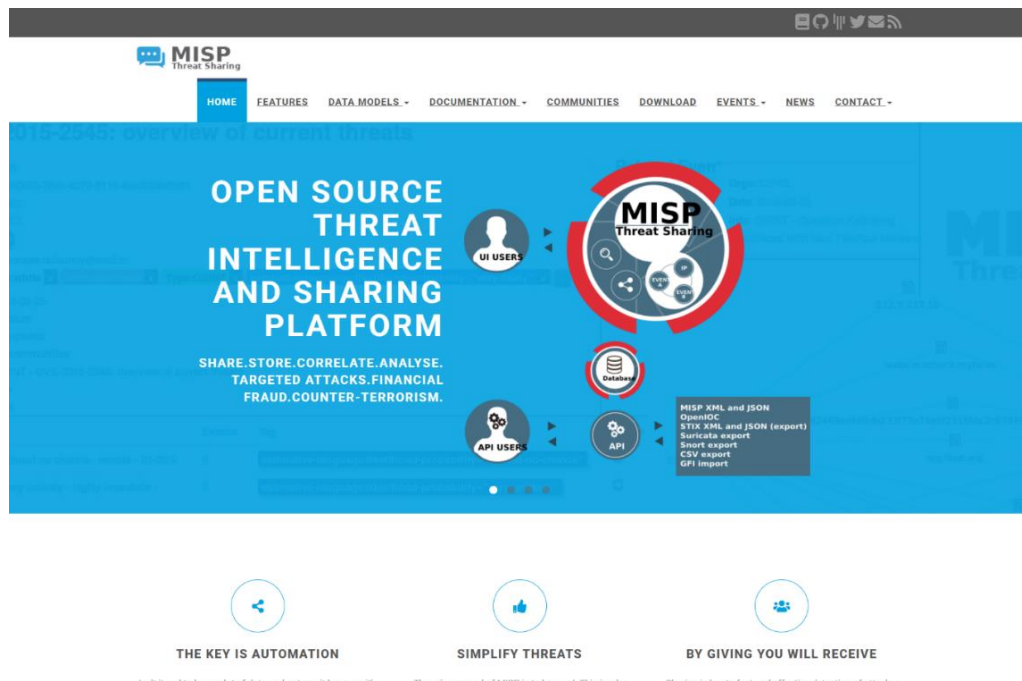


Figure 12: MISP website main page.

It supports real-time synchronization among instances, sharing controls, metadata tagging, and enrichment modules, enabling analysts to track relationships between indicators [30], map to known tactics or threat actors, and build a structured knowledge base that can be shared within or across organizations. [30]

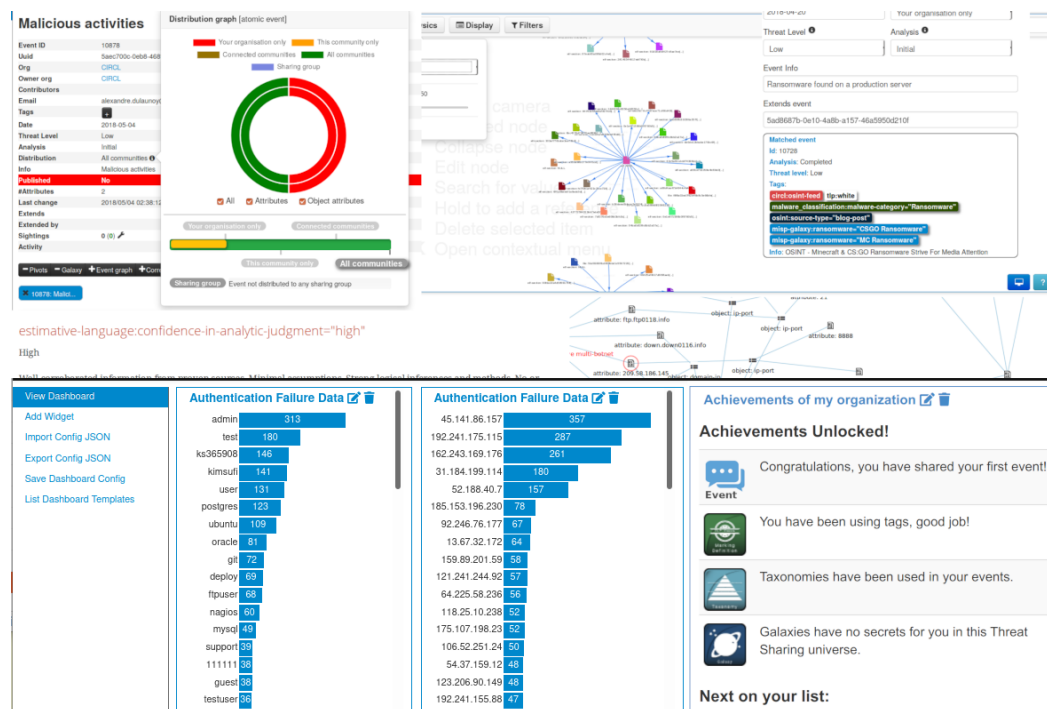


Figure 13: MISP threat-intelligence dashboard showing malicious activity events, visual relationship graphs, authentication-failure statistics, and gamified achievement indicators used to monitor and analyze security incidents.

These capabilities make MISP highly effective for collaborative threat intelligence, community-based IoC exchange, and building a contextualized threat landscape rather than just raw indicators.

2.3.1.2 OpenCTI (Open Cyber Threat Intelligence)

OpenCTI is an open-source, knowledge-graph based threat intelligence platform focused on visualizing, organizing and analyzing cyber threats [31] [31]. It enables the representation of threat actors, campaigns, malware, infrastructure, and IoCs, and the relationships between them in a graph model [32] [32].

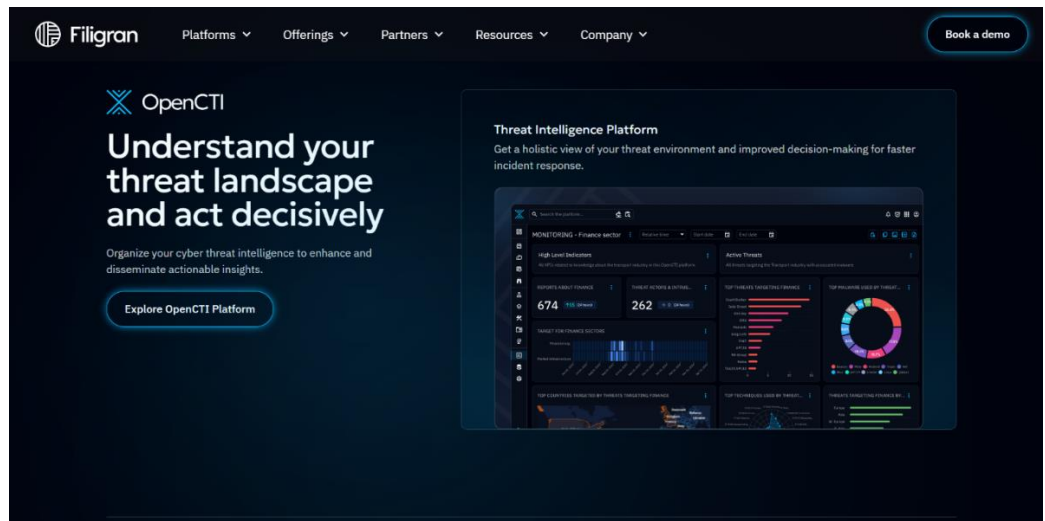


Figure 14: OpenCTI website main page.

By using such a model, OpenCTI allows for high-level reasoning about threats, campaign attribution, TTP mapping, and strategic threat intelligence workflows. Rather than focusing purely on raw IoCs, it provides analytical context, enabling security teams to understand how different indicators relate to adversary behavior and long-term campaigns.



Figure 15: OpenCTI threat-intelligence dashboard displaying high-level incident metrics, geographic distribution of activity, campaign statistics, and visual analytics (bar charts, heatmaps, and donut charts) to help analysts monitor and prioritize emerging threat.

2.3.1.3 FortiAnalyzer

FortiAnalyzer is a commercial security analytics and operations platform developed by Fortinet [33] [33]. It serves as a central hub for log ingestion, normalization, and analytics. FortiAnalyzer consolidates telemetry across networks, endpoints, and cloud environments into a unified data lake. This unified data repository supports built-in SOC automation, AI-assisted threat detection, and real-time correlation of logs and alerts, allowing security teams to obtain a consolidated view of security events, vulnerabilities and suspicious behaviors from a single control point.

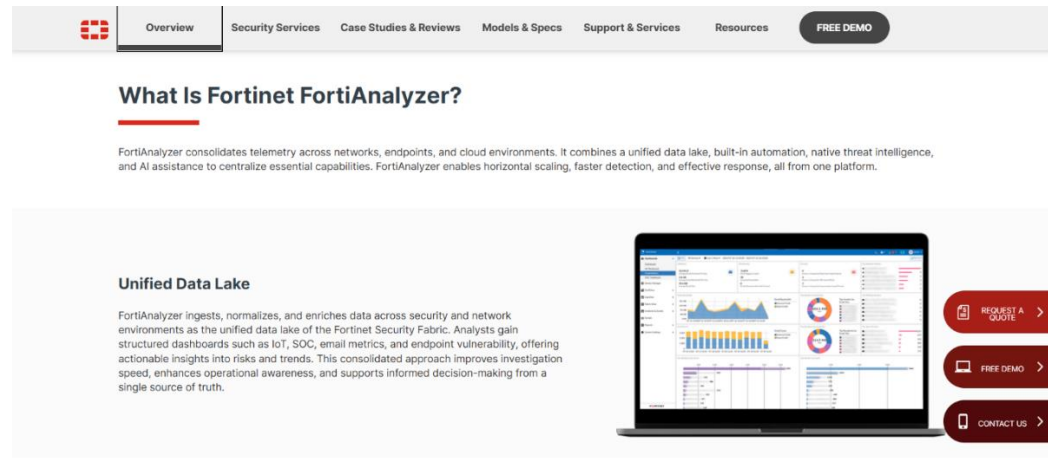


Figure 16: FortiAnalyzer website main page.

The platform's focus is its ecosystem dependency. Because FortiAnalyzer is optimized for Fortinet devices [34] [33], its functionality is extended the more you have an ecosystem of Fortinet products [34].

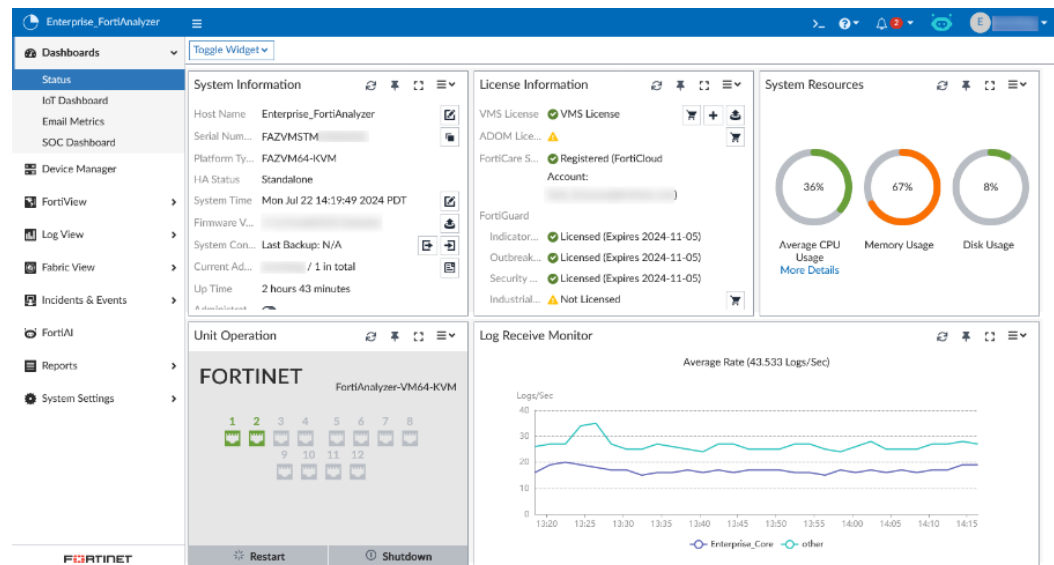


Figure 17: Fortinet FortiAnalyzer system and license information, resource usage, device status, and real-time log ingestion rates.

2.3.2 Issues with the current tools

Although these platforms each provide valuable capabilities in threat-intelligence management or security analytics, they exhibit several limitations when evaluated from the perspective of a multi-scanner orchestration environment such as our project. These shortcomings highlight why existing platforms are insufficient for environments that must centrally manage IoCs, distribute scanner rule files, and collect detection results from servers.

MISP, despite being one of the most widely adopted open-source threat-intelligence sharing platforms, does not provide native support for orchestrating IoC scanners or distributing rule files to servers. While it allows exporting IoCs in multiple formats, the process of deploying these signatures to servers remains manual. It also does not offer mechanisms for discovering hosts, tracking their status, or consolidating detection results. Its focus remains on storing, enriching, and sharing indicators, meaning that organizations must still rely on external workflows or custom automation to operationalize those indicators across scanner environments. Consequently, while MISP excels at intelligence aggregation, it does not address the operational complexity described in Chapter 1, particularly the fragmentation caused by maintaining multiple scanners on multiple machines.

OpenCTI also presents limitations when considered as part of an IoC-driven detection workflow. As a knowledge-graph platform, OpenCTI focuses heavily on modeling relationships between threat actors, campaigns, TTPs, and IoCs. This makes it powerful for strategic and operational threat analysis, but it does not provide tooling for deploying IoC rule files or integrating with scanners. It supports ingestion of external intelligence feeds and provides connectors for bidirectional data exchange, but it lacks components for sending rules to servers, running scans, or retrieving scan results. OpenCTI remains an intelligence-management solution, not a scanner-orchestration system. This results in the same operational gap: even though it enriches and structures intelligence, analysts still must manually translate that intelligence into active scanning and monitoring configurations.

FortiAnalyzer, while strong in log management, event correlation, and SOC analytics, suffers from ecosystem limitations. It is primarily designed for environments built around Fortinet products and does not natively integrate with external IoC scanning engines. Its ingestion pipeline favors FortiGate, FortiEDR, and other Fortinet telemetry sources, which restrict organizations that use open-source or mixed-vendor detection ecosystems. Moreover, FortiAnalyzer does not support distributing IoC rule files to external scanners, nor does it provide management of non-Fortinet scanning servers. As a result, even though it offers powerful dashboards and analytics, it cannot act as a unified control plane for multi-scanner deployments. This makes it unsuitable as a solution for environments that require vendor-agnostic orchestration and centralized rule distribution across systems.

Collectively, these limitations demonstrate that while current tools excel in intelligence management or log analysis, they do not address the operational challenges of multi-scanner IoC deployment. None of them provide mechanisms for automatically distributing rule files, discovering scanner-enabled servers, triggering scans, or aggregating results across scanners and hosts. These gaps reinforce the need for a dedicated IoC Manager designed explicitly to unify IoC ingestion, rule deployment, scanner coordination, and result consolidation into a single operational workflow.

2.3.3 Relationship Between Relevant Work and Our Own Work

While MISP, OpenCTI, and FortiAnalyzer provide capabilities for threat-intelligence storage, enrichment, and analytics, none of them address the operational challenges involved in deploying Indicators of Compromise across scanners and distributed servers. As discussed in Section 2.3.2, current tools focus primarily on intelligence management rather than on orchestrating scanners, distributing rule files, or consolidating detection results. This creates a gap between knowing what indicators exist and actively enforcing them across the environment, a gap our project aims to bridge.

The IoC Manager builds directly on the strengths of existing platforms while addressing the shortcomings. Instead of limiting itself to intelligence enrichment or log correlation, the IoC Manager extends functionality into the execution layer. It provides automated distribution of rulesets to scanning servers, manages hosts through server discovery and status tracking, and consolidates detection results from multiple engines into a unified dashboard. These operational features directly resolve the workflow

fragmentation described in Chapter 1, where analysts must switch between terminals, manually upload rule files, and retrieve results from multiple servers.

A distinguishing contribution of this project is its role as an orchestration layer, one that connects IoC management with actual scanning activity across the network. By enabling coordinated scan execution, real-time result ingestion, and automatic rule synchronization, the IoC Manager reduces manual overhead and improves consistency across scanners. The system remains vendor-neutral and flexible, supporting open-source scanning engines without dependency on any specific commercial ecosystem. As a result, it provides a lighter and more operationally focused alternative to intelligence-heavy platforms such as MISP or OpenCTI, as well as a more scanner-agnostic environment than FortiAnalyzer [31] [33] [29].

To illustrate, we have provided a table that summarizes relevant features and highlights the areas where current tools fall short.

Capability	MISP	OpenCTI	FortiAnalyzer	IoC Manager
IoC Storage & Structuring	✓	✓	✓	✓
Threat Intelligence Correlation	✓	✓	Partial	Limited
Rule File Generation	Partial	✗	✗	✓
Centralized Rule Distribution to Scanners	✗	✗	✗	✓
Management of Distributed Servers / Scanner	✗	✗	✗	✓
Real-time Ingestion of Results	✗	✗	Partial	✓

Vendor-agnostic operation	✓	✓	✗	✓
----------------------------------	---	---	---	---

Table 2: a comparison table of key features across threat intelligence platforms.

The table shows one clear pattern:

- All platforms excel in storage and analysis of IoCs.
- None deploy IoC rules to scanners.
- None collect results from scanners into one single interface.
- None provide server discovery.
- The majority tools are very expensive and paid for services.

These are the capabilities our project aims to implement and fill.

2.4 Summary

In this chapter, we discussed existing work related to IoC management, threat intelligence platforms, and centralized security analytics. While tools such as MISP, OpenCTI, and Fortinet FortiAnalyzer offer adequate capabilities for sharing IoCs, and log analysis, they lack the operational features required to manage distributed scanners in environments. None of these platforms provide centralized rule distribution, multi-scanner orchestration, or unified collection of scanner-generated detection results.

By identifying these shortcomings, this review establishes a clear rationale for the IoC Manager. The system aims to bridge the gap between threat intelligence and operational detection by providing a single platform for ingesting IoCs, distributing rule files, managing scanners, and consolidating results. These insights lay the foundation for the next chapter, which presents the System Analysis of the proposed solution.

3 Chapter 3: System Analysis

3.1 Introduction

System analysis is carried out to understand how the current IoC workflow operates, identify its limitations, and determine what needs to be improved. By examining existing tools, processes, and user needs, this chapter defines the functional and non-functional requirements of the proposed IoC Manager and lays the foundation for its design and implementation.

3.2 Analysis of Existing Systems

In many environments, IoCs are handled using a mixture of threat-intelligence platforms and standalone scanners. Platforms such as MISP and OpenCTI focus on collecting, enriching, and sharing IoCs (for example, IP addresses, domains, file hashes) between teams and organizations. They provide strong capabilities for storing large volumes of indicators, tagging them with context, and exporting them in standard formats. However, they do not directly manage how rule files for scanners like Snort, YARA, or Sigma are pushed to individual servers inside the local network, nor do they provide a unified operational view of scan jobs and matches on those servers.

Vendor-specific products such as Fortinet FortiAnalyzer offer integrated IoC and log-analysis services within a single ecosystem. They can compare historical logs against new IoCs and raise alerts on compromised hosts, but they are tightly coupled to the vendor's appliances. This makes them less suitable when an organization wants to use open-source scanners and rulesets (Snort, YARA, Sigma) deployed on generic servers, or when it wishes to avoid vendor lock-in.

As a result, many security teams still rely on a manual workflow: downloading IoC feeds, editing or generating separate rule files for each tool, copying those files to multiple servers, triggering scans independently, and then collecting results from different consoles or log files. This fragmented approach is time-consuming, error-prone, and makes it difficult to answer simple questions such as “which servers have received the latest rules?” or “where was this IoC last detected?”.

The gap identified from this analysis is the lack of a lightweight, vendor-neutral system that sits between IoC sources and multiple scanners, discovers and tracks scanners in the network, distributes YARA/Snort/Sigma rule files to them, and aggregates their detections into a single dashboard. The proposed IoC Manager is designed to address this specific operational gap rather than replace full threat-intelligence platforms.

3.3 Requirements Elicitation

The requirements of the proposed IoC Manager were derived from the analysis of existing systems and workflows described in Section 3.2, together with the project scope and constraints of a virtual target-servers environment. Since there is no single real-world client, the elicitation relied on document review and observation of typical IoC workflows where analysts manually push IoC rules to multiple scanners and collect results from separate consoles. From this analysis, a set of functional and non-functional requirements was defined to address the identified gaps and guide the design of the IoC Management Process.

3.3.1 Functional Requirements

A. Ingestion & Normalization

A1. The system shall accept IoC and alert data via file upload or API (e.g., Snort alert logs, Sigma/YARA rule files), parse relevant fields (timestamp, SID, message, source IP, destination IP, ports, protocol), map them to internal IoC fields, and store them in the database.

A2. The system shall store raw Sigma rules and their metadata (such as title, ID, author, and tags).

A3. The system shall store YARA rules and any match information (such as rule name, matched strings, and file hashes).

B. IoC Repository & Lifecycle

B1. The system allows users to create, edit, and delete IoCs and rule records via the UI, validating required fields and values before saving.

B2. The system displays a status for each discovered IoC (Verified, Active, Expired and Suppressed).

B3. The system shall maintain a status for each IoC (e.g., Verified, Active, Expired, Suppressed).

B4. The system tags and classifies the IoCs if valid (Malware family, CVEs [35] and kill chain [35]) [35].

B5. The system may detect identical or derived indicators and group them together for easier management.

C. Alerting

C1. The system shall display and push notifications for important events (e.g., new detections, failed exports, or critical IoC status changes) to the dashboard.

D. Visualization & Reporting

D1. The system shall generate basic charts and lists (e.g., top attacker IPs, recent matches, IoCs by status or severity) based on data stored in the IoC Manager database.

D2. The system allows users to export filtered IoC and detection data (by type, tags, status, time range) in common formats such as CSV, JSON, or TXT.

E. IoC Scanners Handling

E1. The system shall collect detection results from the various IoC scanners used and link each result to the relevant IoC, rule, and server.

F. Search

F1. The system should provide indexed search and filters over IoCs and detections (e.g., by time, type, tags, scanner, server, or status).

G. Governance and RBAC

G1. The system shall enforce role and permission-based checks on sensitive actions (such as creating, editing, deleting IoCs or rules, and exporting rule files).

G2. The system shall log key user and system actions (such as logins, rule changes, exports, and deletions) and allow basic viewing or export of these logs.

H. Data Management

H1. The system shall provide mechanisms to purge or archive older IoC and detection records according to simple, configurable retention settings.

I. Servers Interaction and Communication

I1. The system shall be able to discover servers within a given IP range using ICMP echo (ping) and record which hosts respond.

I2. The system shall be able to distribute rule files and, where needed, scripts/commands to target hosts over SSH.

J. Authentication

J1. The system shall provide a registration or admin user-management UI to create user accounts (name, username or email, password) and validate them.

J2. The system should authenticate users at login and restrict access to IoC Manager functions to authenticated users.

3.3.2 Non-Functional Requirements

Non-Functional Requirements	Specifications
Faultlessness	• Successful ingestion rate for valid IoCs $\geq$ 98.0% (hash/IP/URL, YARA/Sigma/Snort data). • Input validation rejects malformed IoCs with clear errors 99% of the time. • An accuracy of $\geq$ 90% in identifying active servers (server sweep), retrieving correct IoC files and distributing scanner rules.
Availability	• The service uptime must be $\geq$ 99.0% monthly. • Response time $\leq$ 500 ms at normal load. • Error rate $\leq$ 5% at normal load.
Fault Tolerance	• If any scanner component/connector (YARA/Sigma/Snort etc.) is down, the manager stays up and queues jobs. • Retry with exponential backoff (periods) with a maximum of 5 tries and then raise an alert. • Job queue/store persists tasks across restarts.

Recoverability	• RTO ≤ 15 min • RPO ≤ 10 min • MTTR ≤ 10 min for unplanned outages. • Configuration must be saved after every change.
Confidentiality	• All network traffic uses TLS (When Available) for manager↔connectors. • Data at rest (Database snapshots) encrypted; keys have scheduled rotation. • RBAC and least privilege, only authorized roles can view/export IoCs, rules, or alerts.
Integrity	• Data should remain accurate and unaltered without permission. • IoC and rule files are versioned and shipped with SHA-256 checksum; verify on ingest & before distribution; reject on mismatch. • Append-only, tamper-evident audit log. • Database integrity: Foreign key / unique constraints.
Non-Repudiation	• Actions performed during an incident should be traceable, ensuring no one can deny their involvement. • Every privileged action is signed/logged with user/service id, timestamp and request id.
Accountability	• Every individual should be held responsible for their actions, and all activities should be traceable. • Unique identities for users and connectors. • RBAC roles with scoped permissions.
Authentication	• The identity of individuals accessing the system should be verified to ensure they are who they claim to be. • MFA required for admin/privileged roles. • Session management: idle timeout 30 min; absolute session age 12 h. • Account lockout / CAPTCHA after repeated failed logins, password policy enforced for local accounts.

Resistance	• The system should be able to withstand attacks or unauthorized attempts. • Rate limiting APIs. • Strong input validation for IoC fields; safe parsers for YARA/Sigma uploads; file size/type limits.
Time Behavior	• Average search query response time: ≤ 3 seconds • Maximum search response time under load: ≤ 5 seconds • CSV upload and validation (10,000 records): ≤ 90 seconds • Export filtered results (up to 10,000 IOCs): ≤ 15 seconds • System startup time: ≤ 45 seconds • Basic enrichment process (10,000 IOCs): ≤ 7 minutes
Space Behavior	• Average memory usage during normal operation: ≤ 2 GB RAM • Peak CPU utilization during import or enrichment: $\leq 70\%$ • Database storage size: ≈ 400 MB per 100,000 IOCs • Disk space required for full system installation and logs: ≤ 5 GB

Table 3: a table that shows the Non-Functional requirements and their specifications.

3.3.2.1 Non-Functional Requirements Modelling (using PiStar)

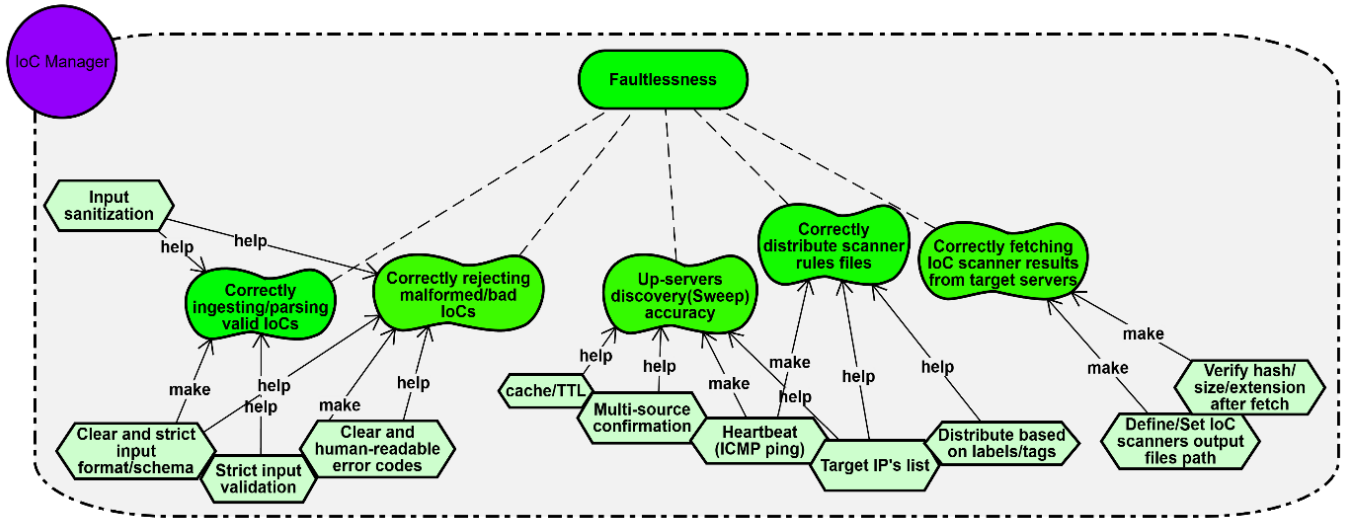


Figure 18: “Faultlessness” reliability model, linking the high-level goal of “Faultlessness” to its qualities and the concrete tasks and mechanisms that make or help achieve each quality.

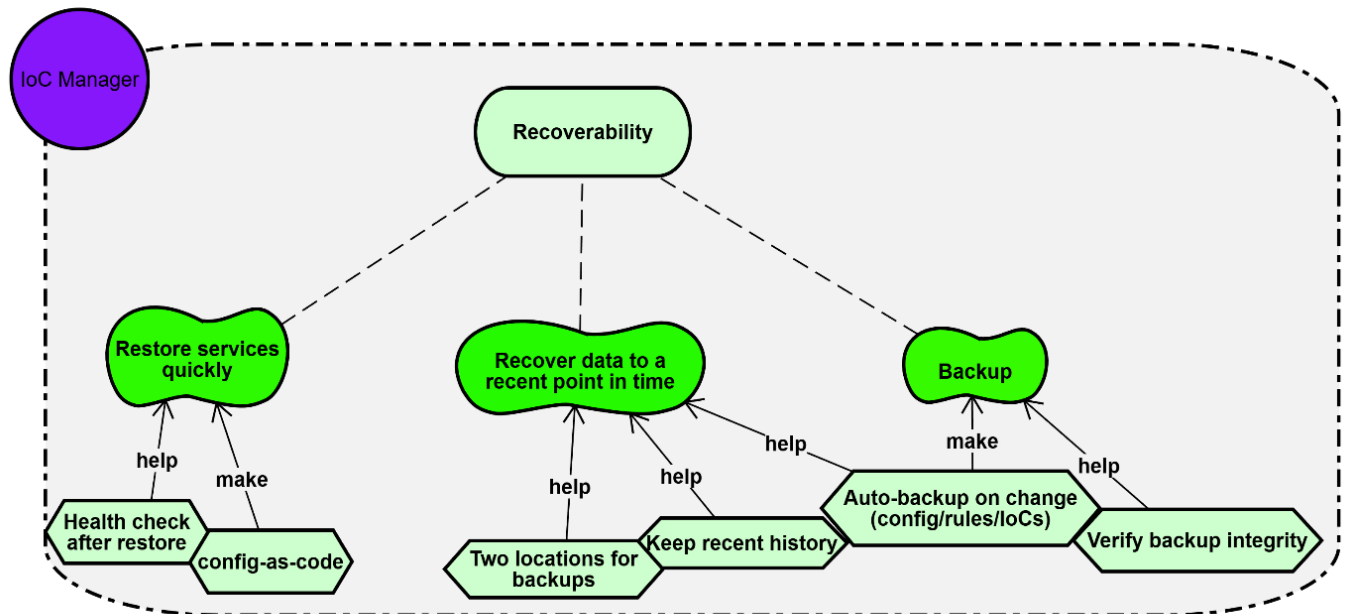


Figure 19: “Recoverability” reliability model, linking the high-level goal of “Recoverability” to its qualities and the concrete tasks and mechanisms that make or help achieve each quality.

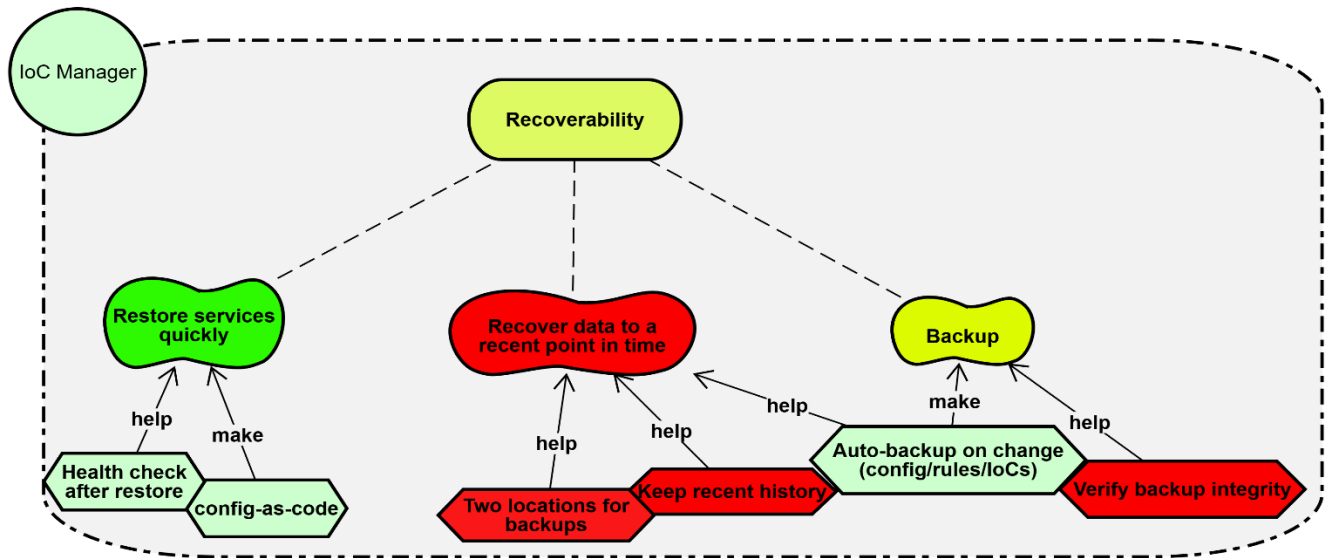


Figure 20: “Recoverability” reliability model illustrating failure scenarios (e.g., Verify backup integrity) and how they impact the recoverability quality attribute.

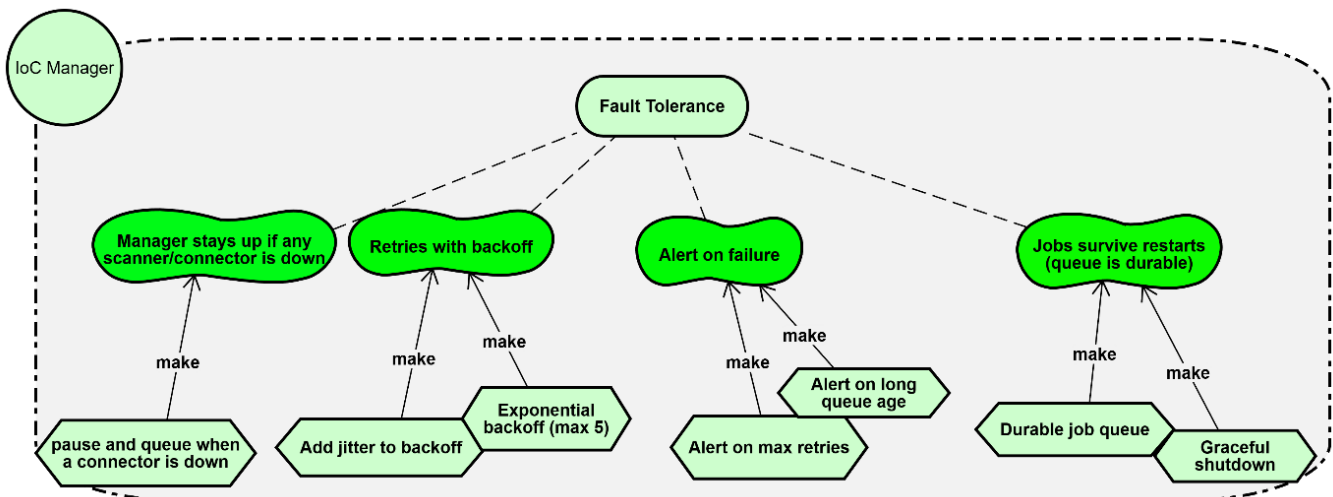


Figure 21: “Fault Tolerance” reliability model, linking the high-level goal of “Fault tolerance” to its qualities and the concrete tasks and mechanisms that make or help achieve each quality.

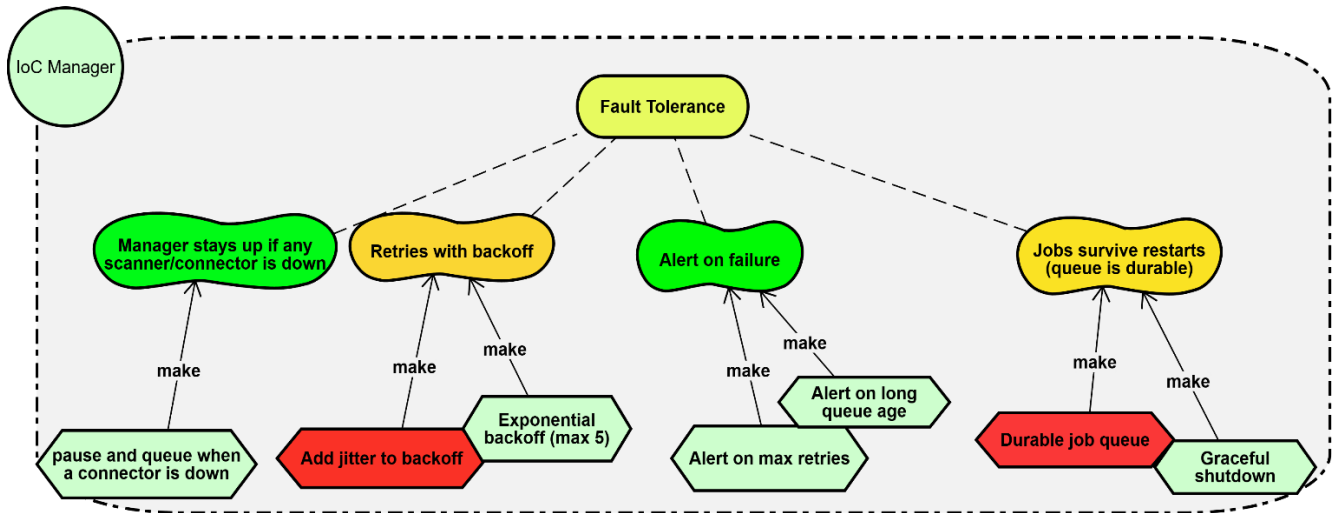


Figure 22: “Fault Tolerance” reliability model illustrating failure scenarios (e.g., failed/queued jobs) and how they impact the fault tolerance quality attribute.

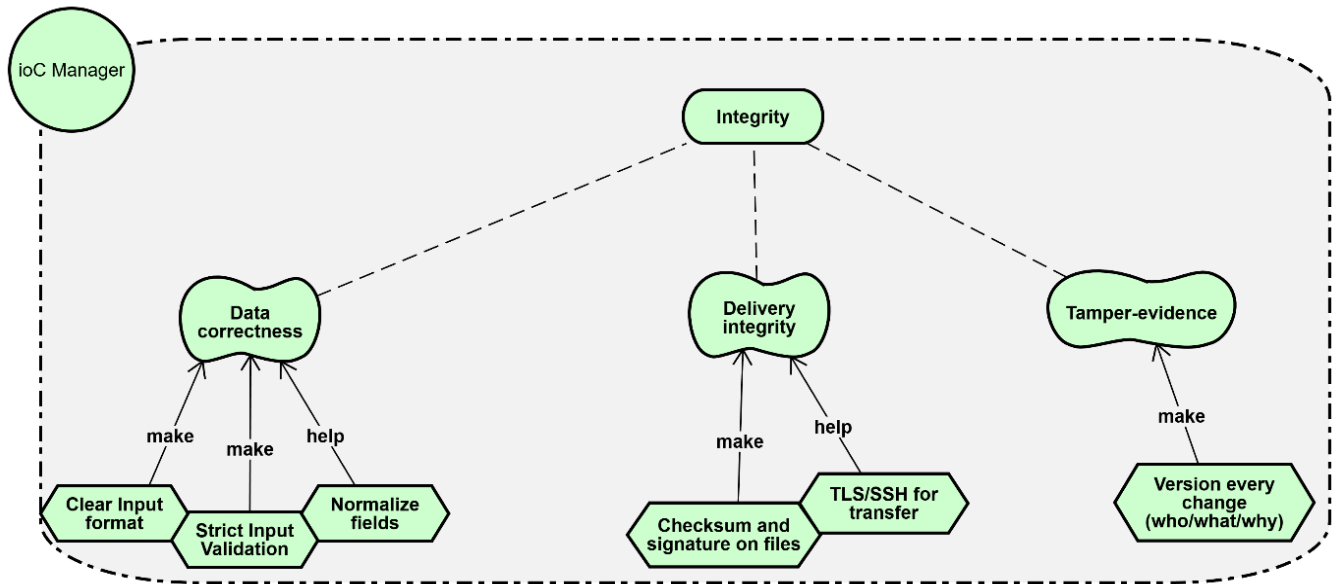


Figure 23: “Integrity” security model, linking the high-level goal of “Integrity” to its qualities and the concrete tasks and mechanisms that make or help achieve each quality.

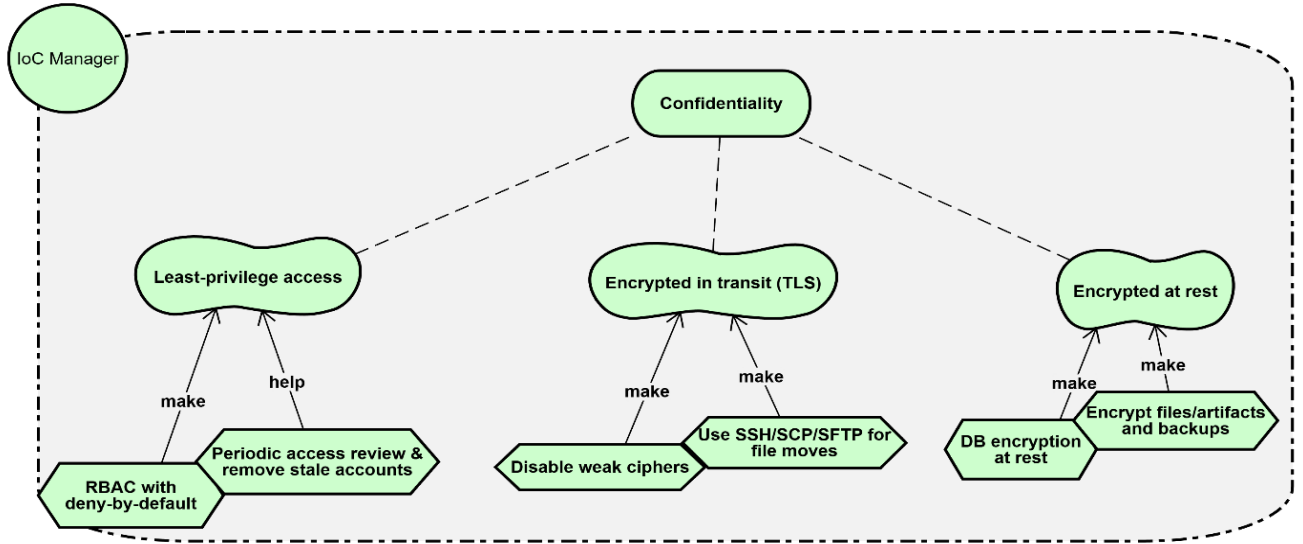


Figure 24: “Confidentiality” security model, linking the high-level goal of “Confidentiality” to its qualities and the concrete tasks and mechanisms that make or help achieve each quality.

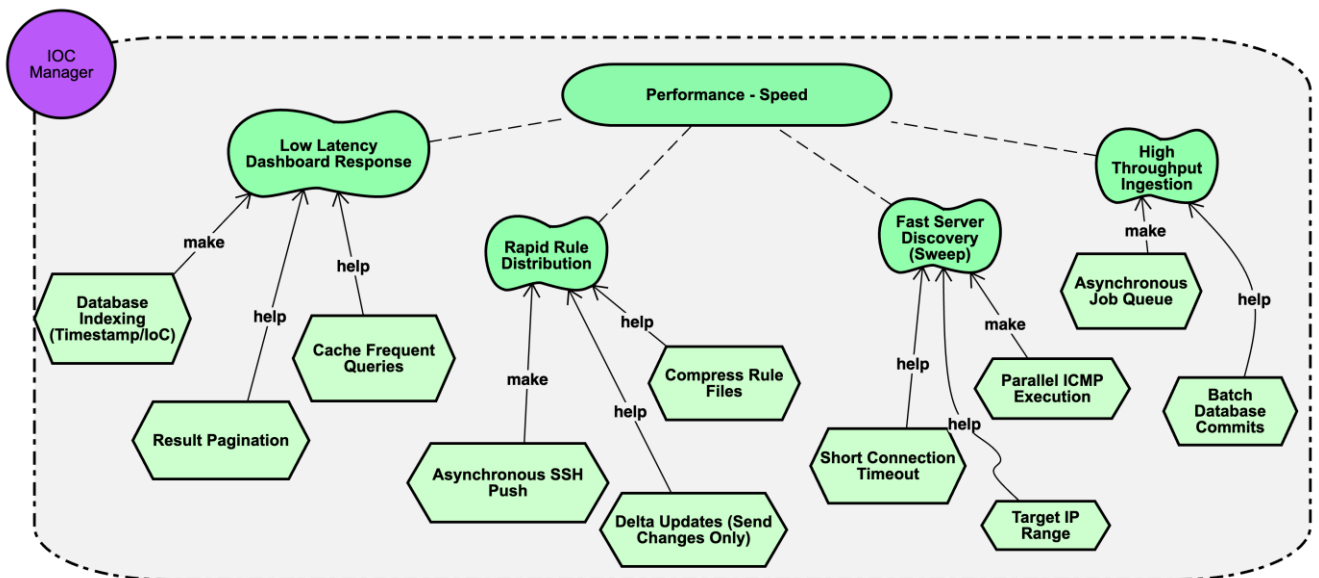


Figure 25: “Performance-speed” model, linking the high-level goal of “Performance-speed” to its qualities and the concrete tasks and mechanisms that make or help achieve each quality.

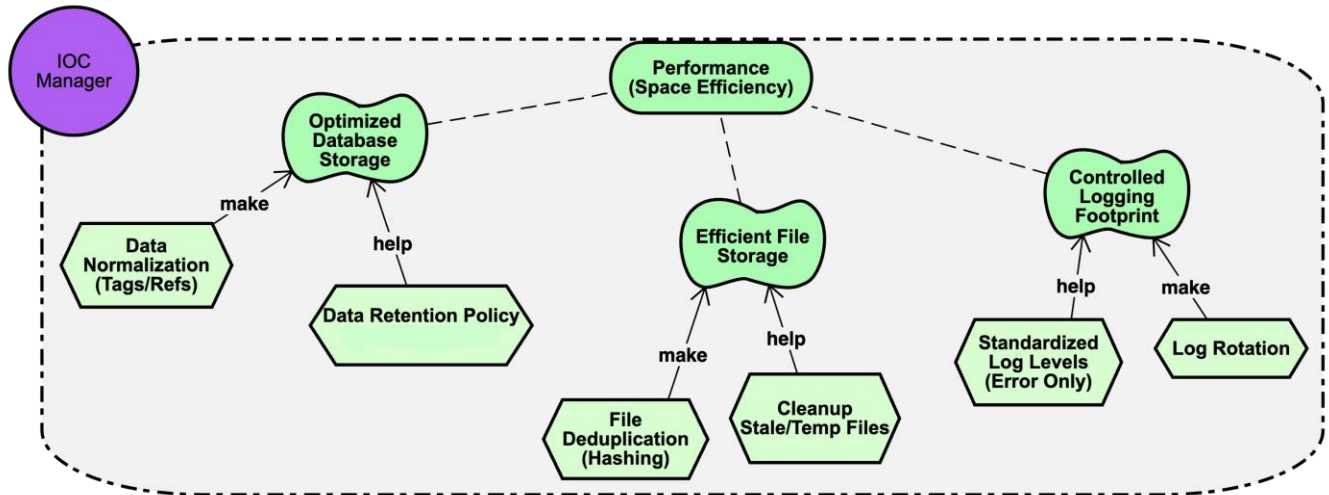


Figure 26: “Performance–space” model, linking the high-level goal of “Performance–space” to its qualities and the concrete tasks and mechanisms that make or help achieve each quality.

3.3.3 User Requirements and Domain Requirements

This section shows the main user-oriented requirements, expressed from the perspective of the intended users of the IoC Manager (Analyst, IT, and Admin), and the domain-specific constraints that shape the system.

3.3.3.1 User Requirements

U1. All users (Admin, Analyst, and IT) shall be able to log in to a single web-based interface and view a dashboard appropriate to their role.

U2. Analysts shall be able to view the main dashboard showing the current status of IoCs, recent detections, and connected servers.

U3. Analysts shall be able to search and filter IoCs and detection events by time range, IoC type, scanner (Snort, YARA, Sigma), status, and server.

U4. Analysts shall be able to open a detection event and view its details, including the matched IoC, the scanner/rule that triggered it, the source and destination information, and the detection time.

U5. Analysts shall be able to receive and view alerts about important events (for example, high-severity detections or failed exports) and mark them as viewed or acknowledged.

U6. IT users shall be able to configure server discovery (for example, define IP ranges) and run network sweeps to identify servers that can host or already run IoC scanners.

U7. IT users shall be able to register and manage discovered servers in the IoC Manager, including basic connection parameters (such as IP address and SSH details).

U8. IT users shall be able to add and manage scan plans (for example, select which rules or rule sets to run on which servers) and trigger Snort, YARA, or Sigma scans individually or on all configured servers.

U9. Admin users shall be able to manage user accounts and roles, set access permissions for the different use cases (such as rule editing, scanning, and server management), and perform maintenance and audit actions on the system (for example, viewing activity logs and configuration changes).

U10. Admin and IT users shall be able to generate and view reports that summarize IoC activity, scan results, and system usage over a selected period.

3.3.3.2 Domain Requirements

D1. The system focuses on network-based and server-side host-based IoCs, such as malicious IP addresses and file hashes observed on servers. Client-side endpoints and email security gateways are outside the scope of this project.

D2. The system shall support Snort, YARA, and Sigma as the primary IoC scanner rule formats. Integration with other proprietary scanners or EDR platforms is considered out of scope for the current prototype.

D3. The system is intended for deployment in an internal, controlled network or lab environment, where servers running Snort/YARA/Sigma are reachable over standard protocols (ICMP for discovery and SSH for rule distribution).

D4. The system shall use a simple username-and-password-based authentication mechanism and role-based access control, without integrating external identity providers (such as SSO) in the current prototype.

D5. The system shall store IoCs, rules, and detection results in a central database so that they can be queried, searched, and reported on without direct access to the underlying scanner logs or consoles.

3.4 Requirements Specification

To make the functional requirements more concrete, the behavior of the IoC Manager is described using UML use-case models.

3.4.1 Use Case Diagram

Use-case modelling is used here to specify how different users (Admin, Analyst and IT) interact with the IoC Manager.

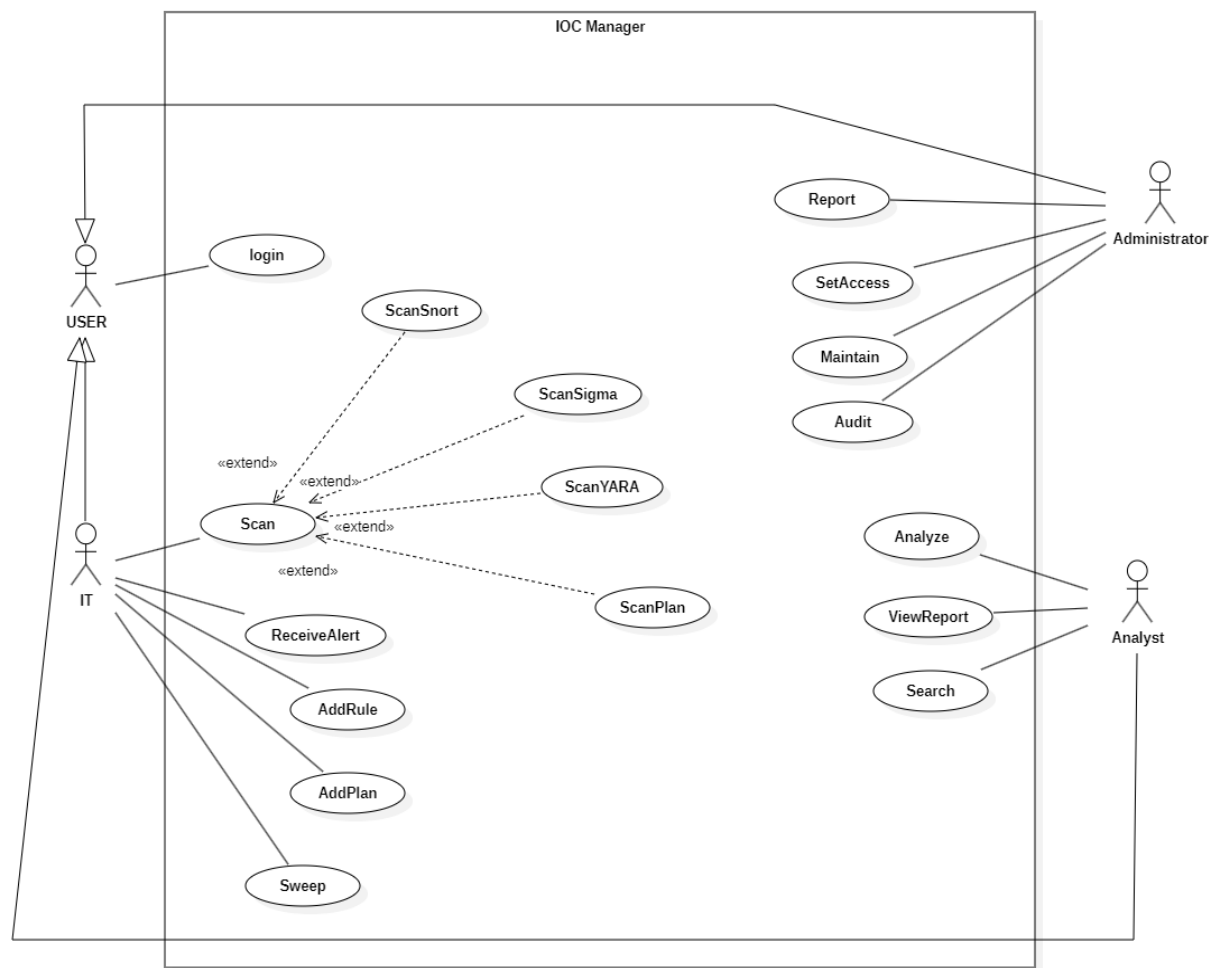


Figure 27: Use-case diagram showing how different actors (IT, Administrator, Analyst) interact with the system to perform various actions on the system.

Use case name	Login
Participating actor	User.
Flow of events	User enters username and password. System authenticates credentials. Users are granted access to IOC Manager.
Entry condition	User enters valid username.
Exit conditions	User sessions are active and authenticated
Quality requirements	Security - The system must secure the session data.

Table 4: “Login” use case description.

Use case name	Scan
Participating actor	IT
Flow of events	IT selects “Scan”. System retrieves Scan plan. System executes scan according to scan plan and returns results.
Entry condition	IT is logged in.
Exit conditions	Selected scan completed.
Quality requirements	

Table 5: “Scan” use case description for executing scan plans by the IT actor.

Use case name	ScanSnort
Participating actor	IT
Flow of events	System loads Snort rules. System analyzes network data. System generates Snort-based detection results.
Entry condition	Snort rules exist.
Exit conditions	Snort scan results available.
Quality requirements	

Table 6: “ScanSnort” use case description for Snort-based network scanning.

Use case name	ScanSigma
Participating actor	IT
Flow of events	System loads Sigma rules. System analyzes logs for Sigma patterns. System generates a detection event System returns Sigma matches.
Entry condition	Sigma rules exist.
Exit conditions	Sigma scan results available.
Quality requirements	

Table 7: “ScanSigma” use case description for Sigma-based log analysis.

Use case name	ScanYARA
Participating actor	IT
Flow of events	System loads YARA signatures. System scans files or memory. System reports YARA detections.
Entry condition	YARA rules available.
Exit conditions	YARA scan results produced.
Quality requirements	

Table 8: “ScanYARA” use case description for YARA-based file.

Use case name	ReceiveAlert
Participating actor	IT
Flow of events	System detects threat indicators. System sends alert notification to IT. IT views alert details.
Entry condition	Scanning reports exist
Exit conditions	IT is informed of IOC findings.
Quality requirements	

Table 9: “ReceiveAlert” use case description for alert notification and viewing by IT.

Use case name	AddRule
Participating actor	IT
Flow of events	IT selects “Add Rule”. IT inputs rule details (Snort/Sigma/YARA). Rules are stored and applied to future scans.
Entry condition	IT is logged in.
Exit conditions	New IOC rule added.
Quality requirements	

Table 10: “AddRule” use case description for creating new IoC rules.

Use case name	AddPlan
Participating actor	IT
Flow of events	IT creates a scanning plan System validates the plan. System stores the plan for automation.
Entry condition	IT logged in.
Exit conditions	Plan saved and ready for execution.
Quality requirements	

Table 11: “AddPlan” use case description for defining and saving scan plans.

Use case name	ScanPlan
Participating actor	IT
Flow of events	IT choose a scan plan System executes the plan on target servers. Scan results are parsed and stores.
Entry condition	IT logged in & Valid Plan
Exit conditions	The target returned results to the system.
Quality requirements	

Table 12: “ScanPlan” use case description for network sweep operations.

Use case name	Sweep
Participating actor	IT
Flow of events	IT triggers a “Sweep”. System performs across selected network areas. Results shown and logged.
Entry condition	IT logged in.
Exit conditions	Sweep results available.
Quality requirements	

Table 13: “Sweep” use case description for network sweep operations.

Use case name	Report
Participating actor	Admin
Flow of events	Admin requests a report. System compiles data (scans, alerts, logs). System generates reports in chosen format.
Entry condition	Admin logged in and scan report exists.
Exit conditions	Admin receives structured report.
Quality requirements	

Table 14: “Report” use case description for generating structured scan and alert reports.

Use case name	SetAccess
Participating actor	Admin
Flow of events	Admin opens access control panel. Admin modifies roles/permissions for users. System updates permission settings.
Entry condition	Admin logged in.
Exit conditions	Updated access control in effect.
Quality requirements	

Table 15: “SetAccess” use case description for managing user roles and permissions.

Use case name	Maintain
Participating actor	Admin
Flow of events	Admin selects maintenance operations. System performs cleanup, optimization, or updates. System confirms action.
Entry condition	Admin logged in
Exit conditions	System maintained.
Quality requirements	

Table 16: “Maintain” use case description for performing system maintenance operations.

Use case name	Audit
Participating actor	Admin
Flow of events	Admin selects “Audit”. System retrieves audit logs. System displays logs.
Entry condition	System audit logs exist.
Exit conditions	Admin reviews audit.
Quality requirements	

Table 17: “Audit” use case description for reviewing system audit logs.

Use case name	Analyze
Participating actor	Analyst
Flow of events	Analyst selects “Analyze”. System shows indicators, matches and alerts. Analyst performs review.
Entry condition	Scan results exist.
Exit conditions	Threat analysis completed.
Quality requirements	

Table 18: “Analyze” use case description for analyst review of indicators, matches, and alerts.

Use case name	ViewReport
Participating actor	Analyst
Flow of events	Analyst selects a report. System retrieves and displays it.
Entry condition	Reports exist.
Exit conditions	Analyst views report.
Quality requirements	

Table 19: “ViewReport” use case description for viewing generated reports.

Use case name	Search
Participating actor	Analyst
Flow of events	Analyst enters search criteria System queries data. System returns filtered results.
Entry condition	User logged in.
Exit conditions	Search results available.
Quality requirements	

Table 20: “Search” use case description for querying and filtering data.

3.5 Developmental Methodology

The IoC Manager will be developed using the **Agile–Scrum methodology**, which will allow the project to be built step by step through short development cycles called sprints. This approach will help the team adapt to changes, review progress regularly, and ensure that each part of the system works correctly before moving to the next.



Figure 28: “Agile development methodology”.

3.5.1 Working Procedure

The project will be divided into several sprints, and each sprint will focus on delivering specific features of the IoC Manager. The development process will follow these steps:

1. **Backlog Preparation**

A product backlog will be created containing all functional and non-functional requirements described in earlier sections. Items will include server discovery, IoC ingestion, rule distribution, dashboard elements, authentication, and reporting.

2. **Sprint Planning**

At the beginning of each sprint, the team will select the features to be developed. Each feature will be broken into smaller tasks so development can be organized clearly.

3. **Sprint Development**

During the sprint, the team will work on implementing the selected tasks. This will include writing code, setting up the components needed, and preparing initial tests.

4. **Follow-Up**

Short meetings will be held to track progress, identify challenges, and coordinate work between team members.

5. **Continuous Testing**

As each feature is completed, it will be tested immediately to ensure it works as expected.

For example, tests will verify:

- a. that network sweep functions detect servers correctly,
- b. that IoC rules (YARA, Snort, Sigma) are loaded and validated,
- c. that detections are stored in the database.

6. **Sprint Review**

At the end of the sprint, the finished work will be demonstrated to review whether the sprint goals were met. Feedback will then be collected.

7. **Sprint Retrospective**

After the sprint review, the team will evaluate what went well, what problems occurred, and what improvements will be applied in the next sprint.

This cycle will continue until all important components of the IoC Manager are developed and integrated.

In addition, the team is managing tasks assignments using Kanban board to draft backlogs tasks and schedule them as planned. The following is a snapshot taken from the GitHub repository of the IoC Manager project page:

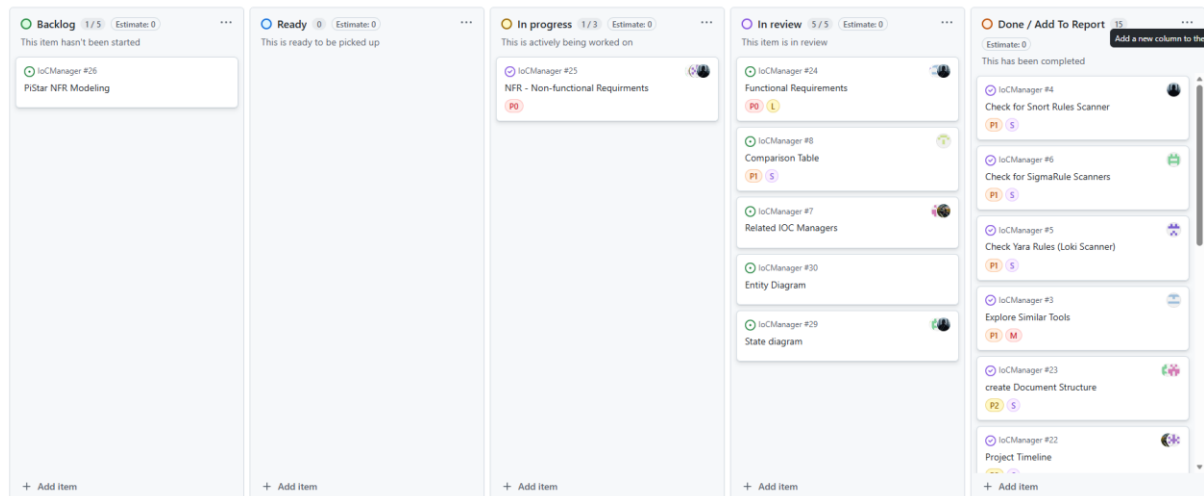


Figure 29: Kanban board illustrating the project's agile task tracking.

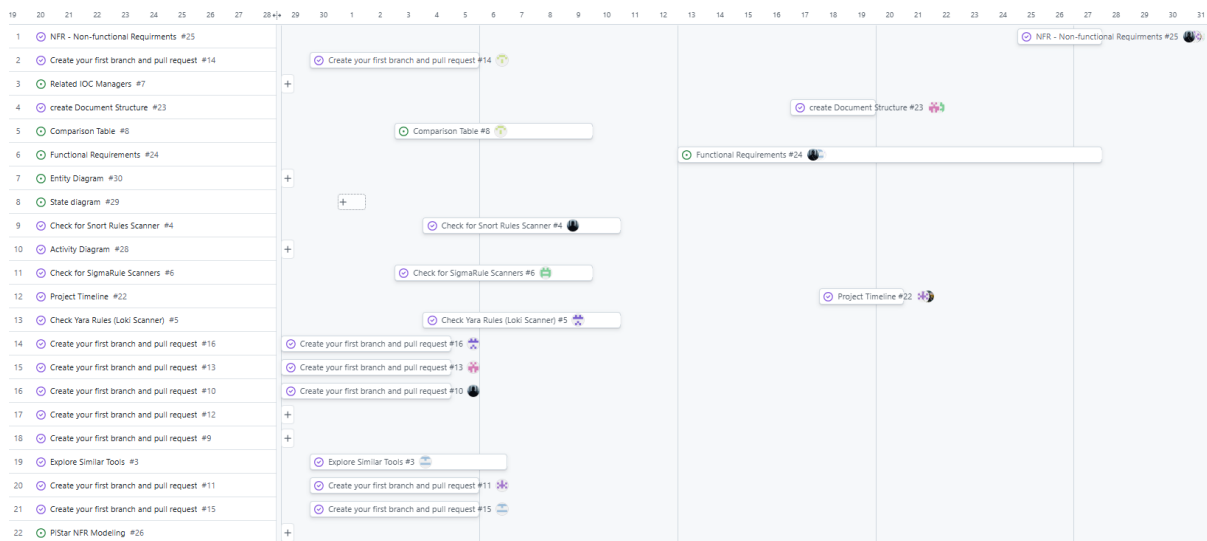


Figure 30: Project timeline view showing scheduled tasks over the sprint.

3.5.2 Conditions and Tools Required

The tools and components described in this report will be used:

- **IoC rule formats:** The system will support industry-standard formats including YARA (file/memory scanning), Snort (network traffic inspection), and Sigma (log/event detection).
- **Scanning components:** The IoC Manager will provide a **sweeper module** and **scanner modules** implemented mainly in .NET (C#). Supporting Python scripts will handle tasks such as log parsing, rule normalization, and result aggregation.

For automation close to the operating system, **Bash scripts** (on Linux targets) and **PowerShell scripts** (on Windows targets) will be used to start/stop scanners, deploy rule files and run command-line operations directly on the target servers. Each target server will have the corresponding scanner engine (Snort, YARA, Sigma-compatible tools) installed and invoked by these modules and scripts.

- **Network communication:** The manager will discover servers using **ICMP ping** and maintain a list of active targets. Rule files and configuration updates will be delivered over secure channels such as **SSH/SCP**, where Bash/PowerShell scripts are executed remotely to apply changes. Management actions and retrieval of scan results may use lightweight **HTTP/HTTPS (REST)** endpoints.
- **Database layer:** Microsoft SQL Server (MSSQL) will be used as the primary relational database to store IoCs, results, alerts and reports, supporting the project's dashboards and queries.
- **Project environment:** Development will be carried out using **Visual Studio Code** as the primary IDE, with the **.NET SDK** for backend services and **Python** for scripting utilities. The runtime environment consists of a **controlled virtual network** with one IoC Manager server and several scanner servers running either Linux (Usually Bash shell) or Windows (PowerShell). This combination of technologies provides both a graphical management layer and low-level command-line automation suitable for implementing and testing all planned features.

3.5.3 Project Constraints

The design and implementation of the IoC Manager are subject to the following constraints:

- **Technical stack constraint:** The backend will be implemented in **.NET (C#)** with **Python** used for auxiliary scripting.
- **Environment and infrastructure constraint:** All development and testing occur in a **controlled virtual lab network** with a limited number of manager and scanner servers.
- **Tooling constraint:** The system will not support every possible IoC / rule technology or vendor product, and rule syntax must remain compatible with these chosen engines.

- **Data and IoC constraint:** For safety and policy reasons, the project uses **synthetic or sanitized IoCs and logs**. Live malware samples, real organization logs, or production threat-intel feeds will not be ingested, which limits the realism of some test scenarios.
- **Network and protocol constraint:** Server discovery and rule distribution will operate on **ICMP, SSH/SCP, and lightweight HTTP/HTTPS APIs** inside the lab network.
- **Time and resource constraint:** The project is developed within a **single academic year**, with limited hardware resources. As a result, performance tuning, horizontal scalability are demonstrated only at a prototype level rather than production-grade scale.

3.6 Data Collecting and Processing

The data used in this project consists mainly of **IoC definitions** and **scanner output logs**. IoCs are prepared either by manually defining test rules or by selecting and sanitizing rules from public **YARA, Snort, and Sigma** rule sets. These rules are uploaded to the IoC Manager through the web interface, where they are stored in the system database together with metadata such as rule type, description, source, and assigned target servers.

For **data collection from targets**, each scanner server runs the corresponding engine (Snort, YARA, Sigma-compatible tool) and executes the rules distributed by the IoC Manager. When a scan is performed, the scanners produce local log files and alert outputs. The results are then returned to the IoC Manager via the configured communication channels (e.g., SSH/SCP for file transfer or simple HTTP endpoints).

Within the IoC Manager, the **processing pipeline** validates and normalizes all incoming data to a common internal schema, regardless of whether it originated from YARA, Snort, or Sigma. Duplicate entries are filtered, malformed records are rejected with clear error messages, and accepted events are stored in the project's relational database. This processed data is then used to populate dashboards and reports, including per-server detection histories and summaries of IoCs across the environment.

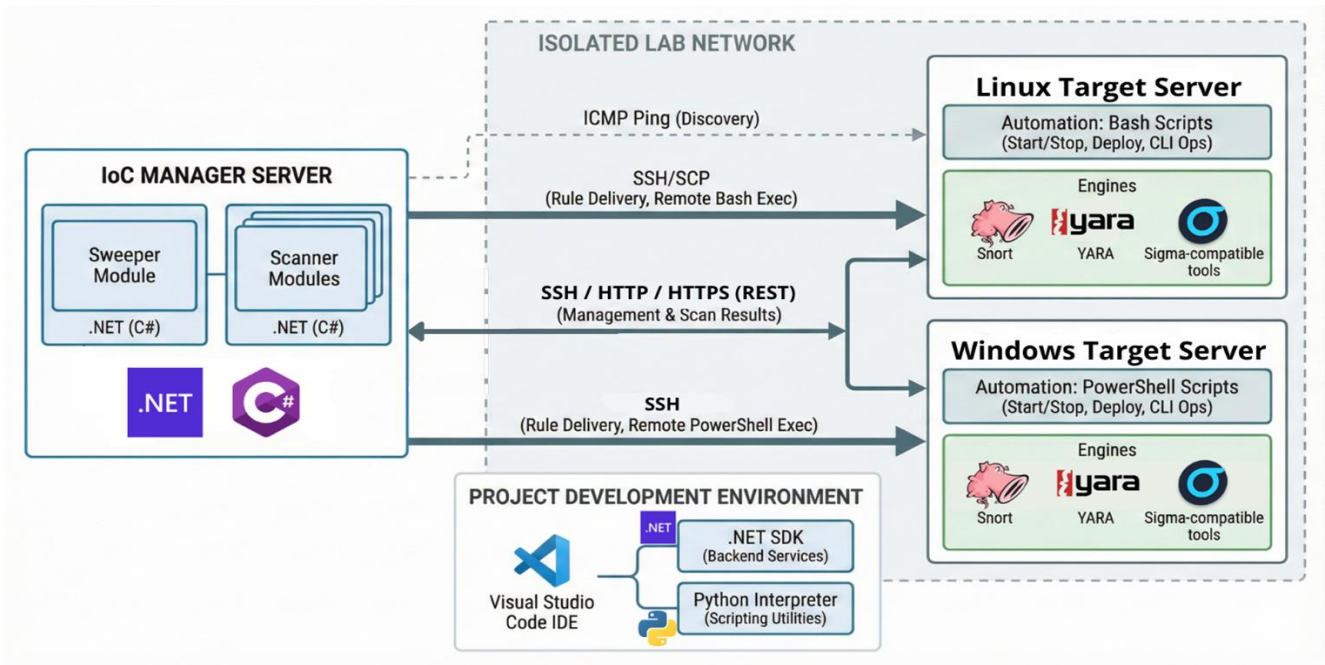


Figure 31: Visual overview of the IoC Manager environment and project constraints.

3.7 Summary

In this chapter, the system analysis for the IoC Manager was presented. The chapter began in **Section 3.2**, where the limitations of existing IoC tools and workflows were examined. This analysis helped identify the gaps that the new system needed to address.

Based on these findings, **Section 3.3** defined the system's functional requirements, non-functional requirements, and user and domain requirements. These requirements described what the IoC Manager needed to achieve and the constraints under which it would operate.

In **Section 3.4**, the requirements were translated into formal specifications through use-case modelling. The use cases clarified how Admin, IT, and Analyst users would interact with the system and supported the behaviors needed to meet the requirements identified earlier in **3.3**.

The methodology used to guide the development of the system was described in **Section 3.5**, where the Agile-Scrum process and sprint structure were outlined. This methodology explained how the system would be developed step by step.

Finally, **Section 3.6** described the data collection and processing activities related to the IoC files, targets, servers and scanning modules. These workflows supported the requirements defined earlier and prepared the foundation for system design.

Overall, this chapter established the analytical basis needed for the design work presented in Chapter 4.

4 Chapter 4: System Design

4.1 Introduction

This chapter describes the overall design of the IoC Manager system based on the requirements defined in Chapter 3. It begins by outlining the architectural view of the solution, explaining how the IoC Manager is structured internally and how it interacts with external scanners and servers. It then presents the main object-oriented models and data models that define the system's core entities and their relationships, followed by the key user interface screens that support administrative workflows. In addition, this chapter summarizes the principal backend workflows and algorithms that turn the design into an implementable solution.

4.2 Architectural Design

The architectural design establishes a centralized command structure aimed at managing distributed operations efficiently. The core objective is to create a system capable of defining rules and configurations centrally and distributing them to various external servers for execution (such as tests or scans).

The proposed architecture is centered around the "IoC Manager." This manager is designed internally using a three-layer architecture to separate concerns between the user interface, business logic (controller and scanner), and data storage. Externally, the IOC Manager functions as the central hub in a one-to-many client-server relationship, issuing commands to dispersed servers and aggregating the returned results for analysis and reporting. This design ensures centralized control while leveraging distributed resources for execution

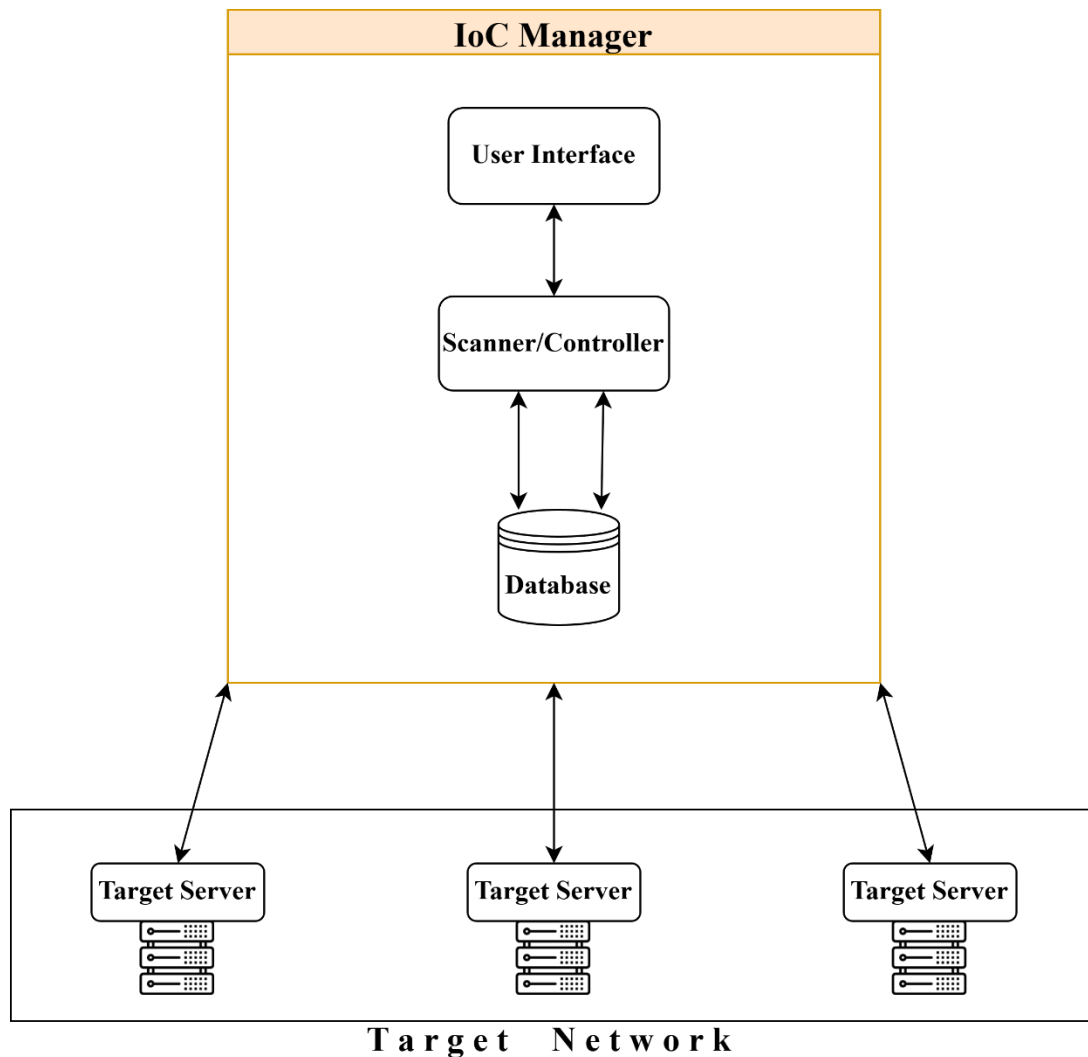


Figure 32: High-level architectural design of our IoC Manager.

4.3 Object-Oriented Design

This section presents the system from an object-level perspective by identifying the main classes, their attributes and procedures and the relationships between them. This design translates the functional requirements into reusable software components. The object-oriented view strengthens modularity by organizing the system into clear responsibilities and well-defined interactions.

4.3.1 Structural Static Models

4.3.1.1 Class Diagram

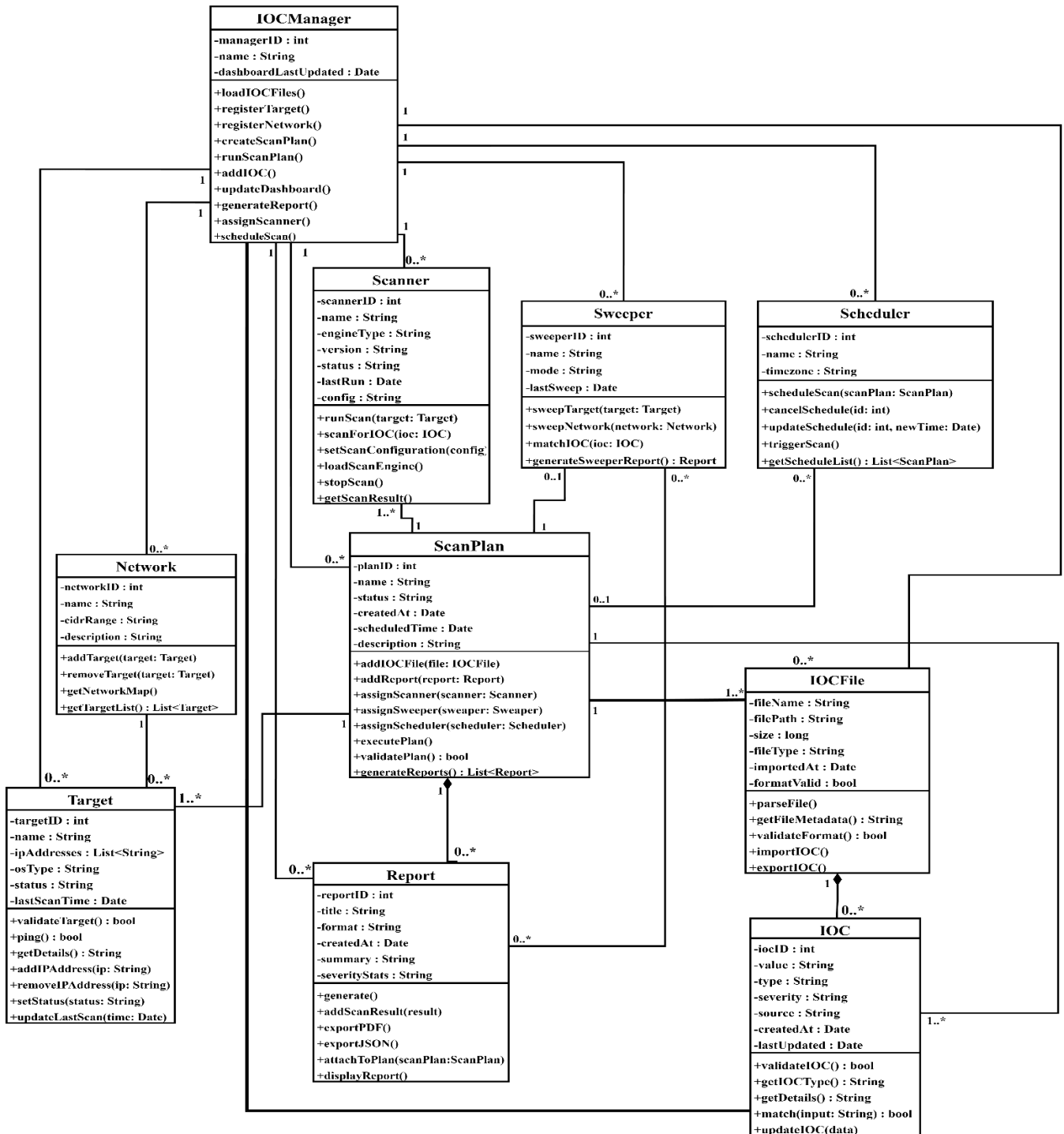


Figure 33: Class diagram of the IoC Manager, showing its main internal objects (Scanner, Sweeper, Scheduler, Network/Target, ScanPlan, IOCFile/IOC, and Report) and how they interact with each other.

Entity	List of Procedures	Data Input	Output Data
IOC Manager	loadIOCFiles()	IOC file(s)	Loaded IOC objects
	registerTarget()	Target information	Target added confirmation
	registerNetwork()	NetworkId + target list	Network registered
	createScanPlan()	Selected IOCs, targets, and configuration	Scan plan object
	runScanPlan()	Scan plan ID	Scan results
	addIOC()	IOC data	IOC added confirmation
	updateDashboard()	System statistics	Updated dashboard view
	generateReport()	Scan results	Final report file
	assignScanner()	Scanner details	Scanner linked acknowledgment
	scheduleScan()	Scan plan + time	Scheduled task ID
IOC	validateIOC()	IOC data	Validation result
	getIOCType()	IOC entry	IOC type
	getDetails()	IOC ID	IOC detailed information
	match(input)	Input value	Match result
	updateIOC(data)	Updated IOC	Updated IOC object
IOCFile	parseFile()	Raw IOC file	Parsed file contents
	loadIOCs()	Parsed IOC entries	IOC objects list
	getFileMetadata()	IOC file	Metadata
	validateFormat()	IOC file	Format validation result

	importIOC()	IOC file	Imported IOC objects
	exportIOC()	IOC list	Exported IOC file
Target	validateTarget()	Target information	Validation result
	ping()	Target IP	Reachability status
	getDetails()	Target ID	Target details
	addIPAddress(ip)	IP address	Updated target info
	removeIPAddress(ip)	IP address	Updated target info
	setStatus(status)	New status	Updated status
	updateLastScan(time)	Timestamp	Updated record
Scanner	runScan(target)	Target info	Scan results
	scanForIOC(ioc)	IOC object	IOC-specific result
	setScanConfiguration(config)	Configuration	Updated settings
	loadScanEngine()	Engine type	Engine loaded
	stopScan()	Scan ID	Termination confirmation
	getScanResult()	Scan ID	Final scan result
Network	addTarget(target)	Target object	Updated network list
	removeTarget(target)	Target object	Updated network list
	getNetworkMap()	Network ID	Network map
	getTargetList()	Network ID	Target list
Sweeper	sweepTarget(target)	Target object	Sweep scan result
	sweepNetwork(network)	Network object	Network sweep results
	matchIOC(ioc)	IOC object	Matched results
	generateSweeperReport()	Sweep results	Sweeper report

Scheduler	scheduleScan(scanPlan)	Scan plan	Schedule entry
	cancelSchedule(id)	Schedule ID	Cancellation confirmation
	updateSchedule(id,newTime)	Schedule ID + new time	Updated schedule
	triggerScan()	Schedule ID	Immediate scan result
	getScheduleList()	—	Schedule list
ScanPlan	addIOCFfile(file)	IOC file	Updated plan
	addReport(report)	Report object	Updated plan
	assignScanner(scanner)	Scanner	Scanner assigned
	assignSweeper(sweeper)	Sweeper	Sweeper assigned
	assignScheduler(scheduler)	Scheduler	Scheduler assigned
	executePlan()	Scan plan	Execution results
	validatePlan()	Scan plan	Validation status
	generateReports()	Execution results	Generated reports
Report	generate()	Scan results	Final report
	addScanResult(result)	Scan result	Updated report
	exportPDF()	Report	PDF file
	exportJSON()	Report	JSON file
	attachToPlan(scanPlan)	Scan plan	Linked report
	displayReport()	Report object	Displayed report

Table 21: list of procedures showing entities, their procedures and data input and output.

4.3.1.2 ERD

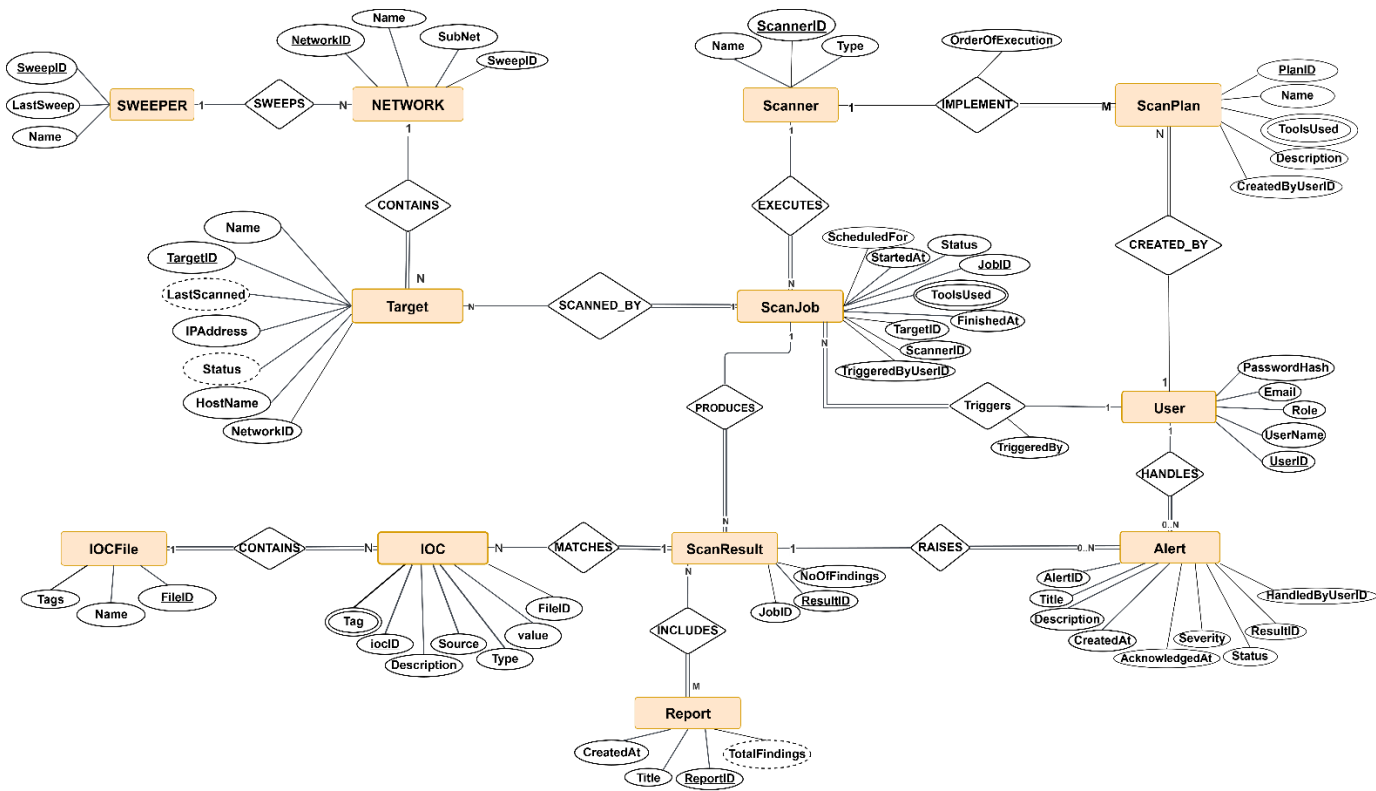


Figure 34: Entity–relationship diagram showing core entities (Sweeper, Network, Target, Scanner, ScanJob, User, Alert, ScanResult, Report, IOCFile, and IOC).

4.3.2 Dynamic Models

This section presents the dynamic behavior of the IoC Manager using a UML state machine diagram, activity and sequence diagram.

4.3.2.1 State Diagram

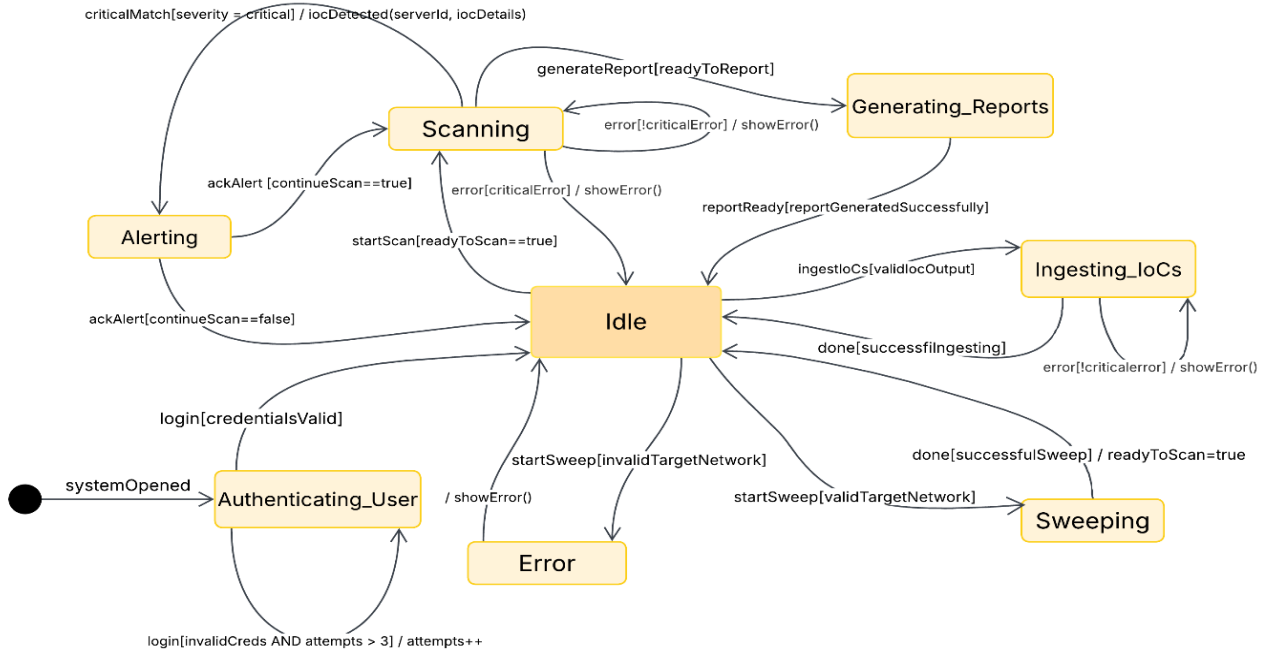


Figure 35: State machine diagram for the IoC Manager, showing how the system transitions between the different system states.

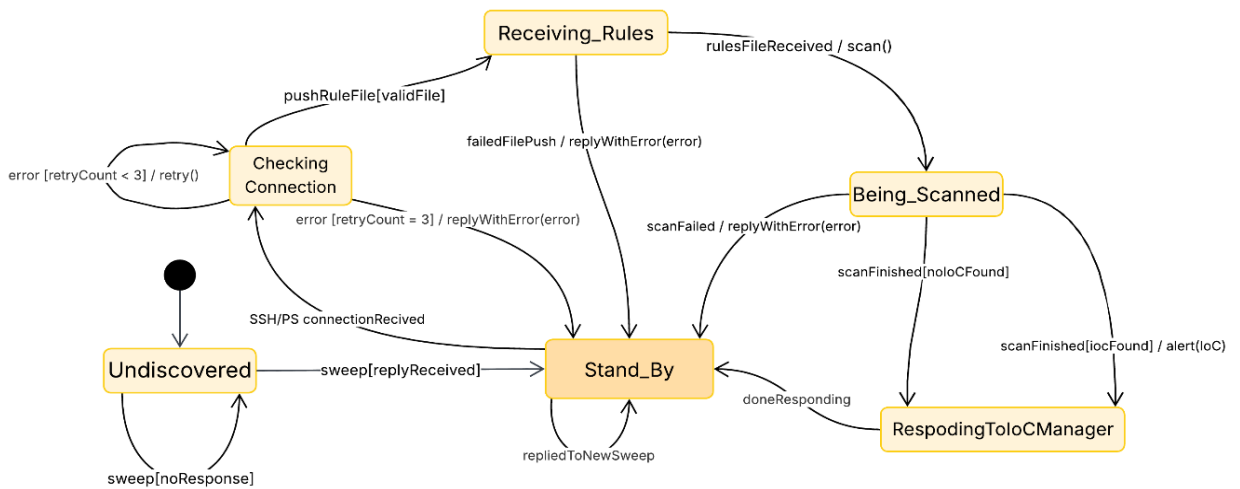


Figure 36: State machine diagram for the target server, showing how the target transitions between the different states.

4.3.2.2 Activity Diagram

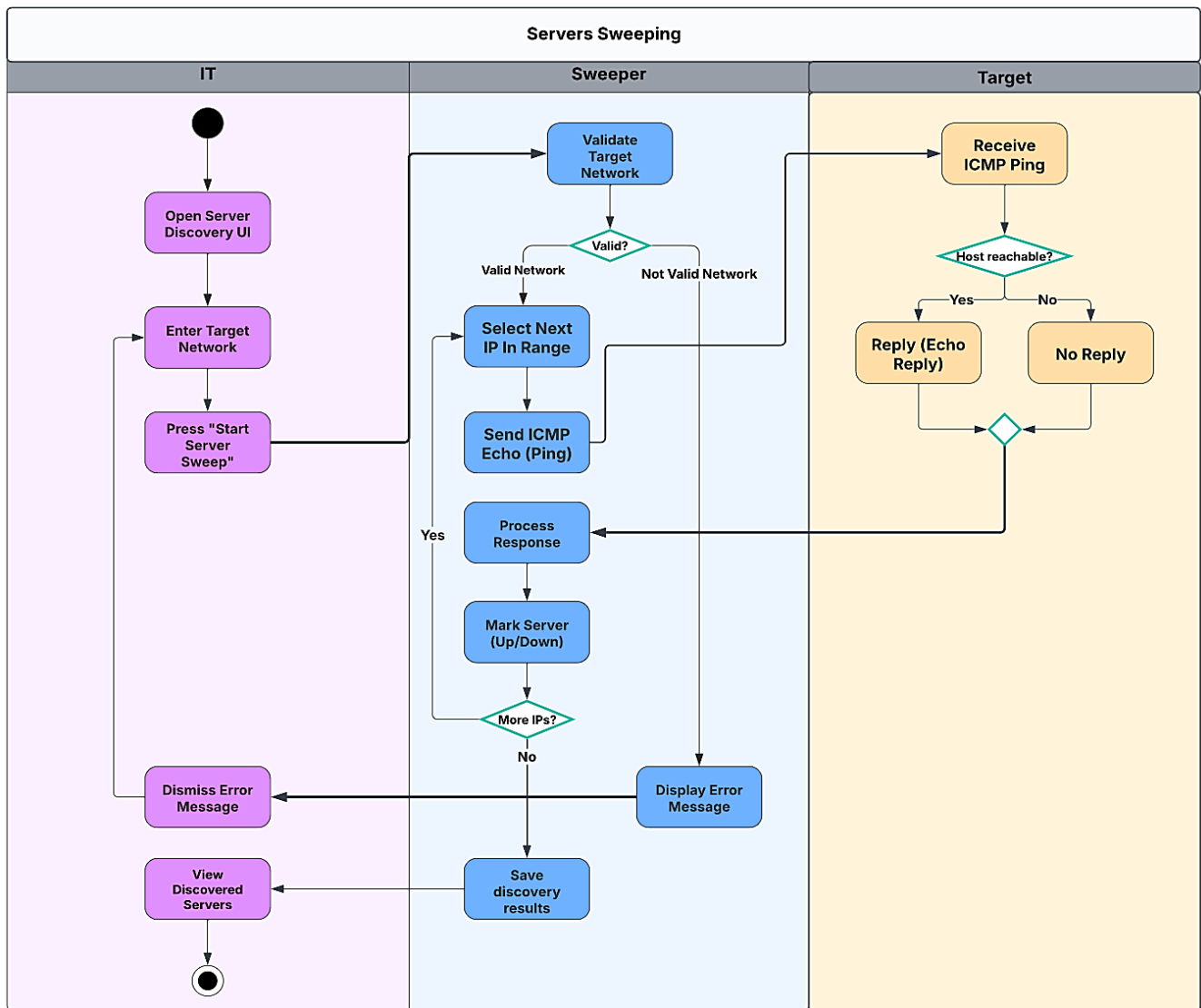


Figure 37: Activity diagram for the server sweeping process.

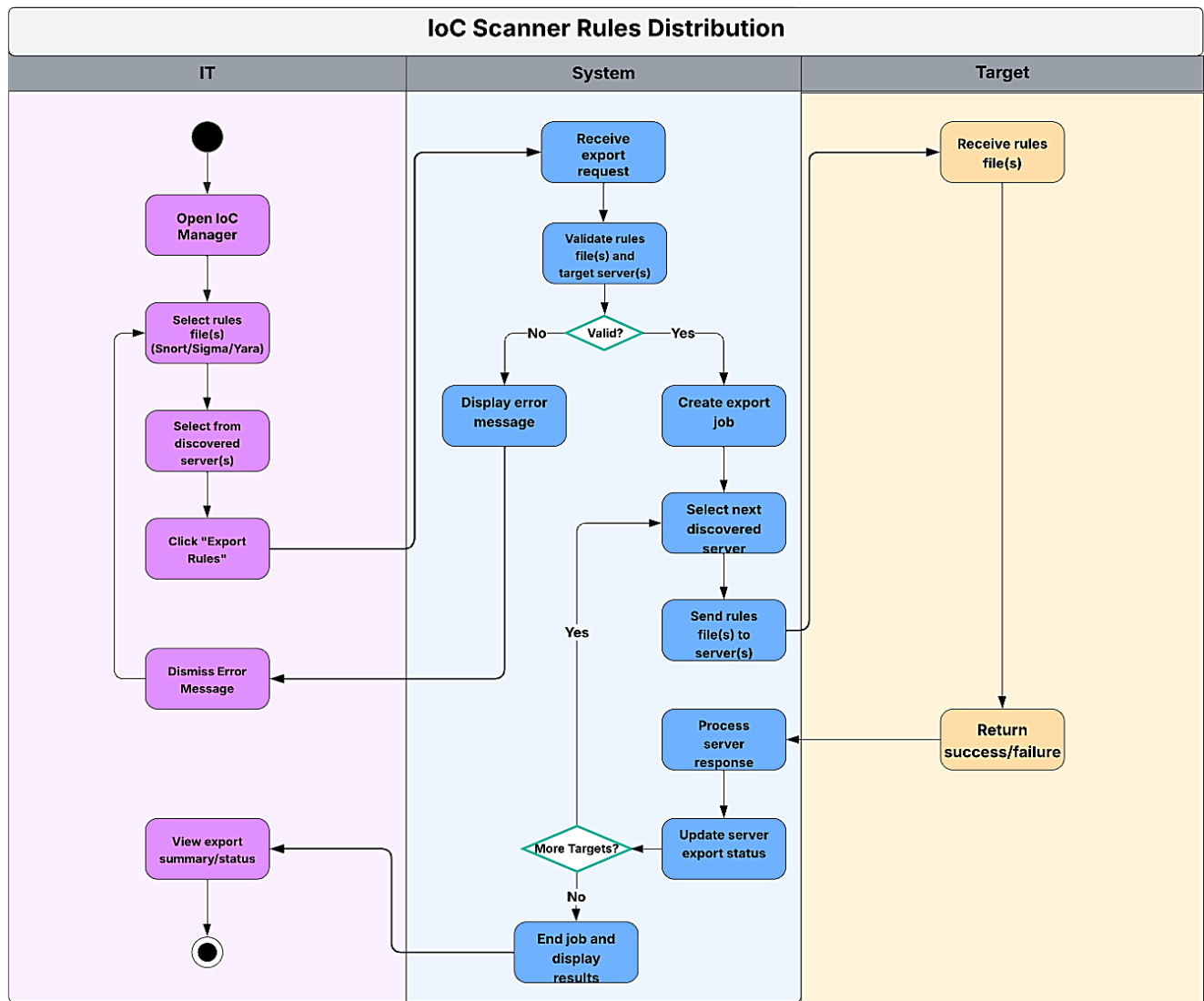


Figure 38: Activity diagram for the IoC scanner rules distribution.

The following diagram shows the high-level threat detection flow:

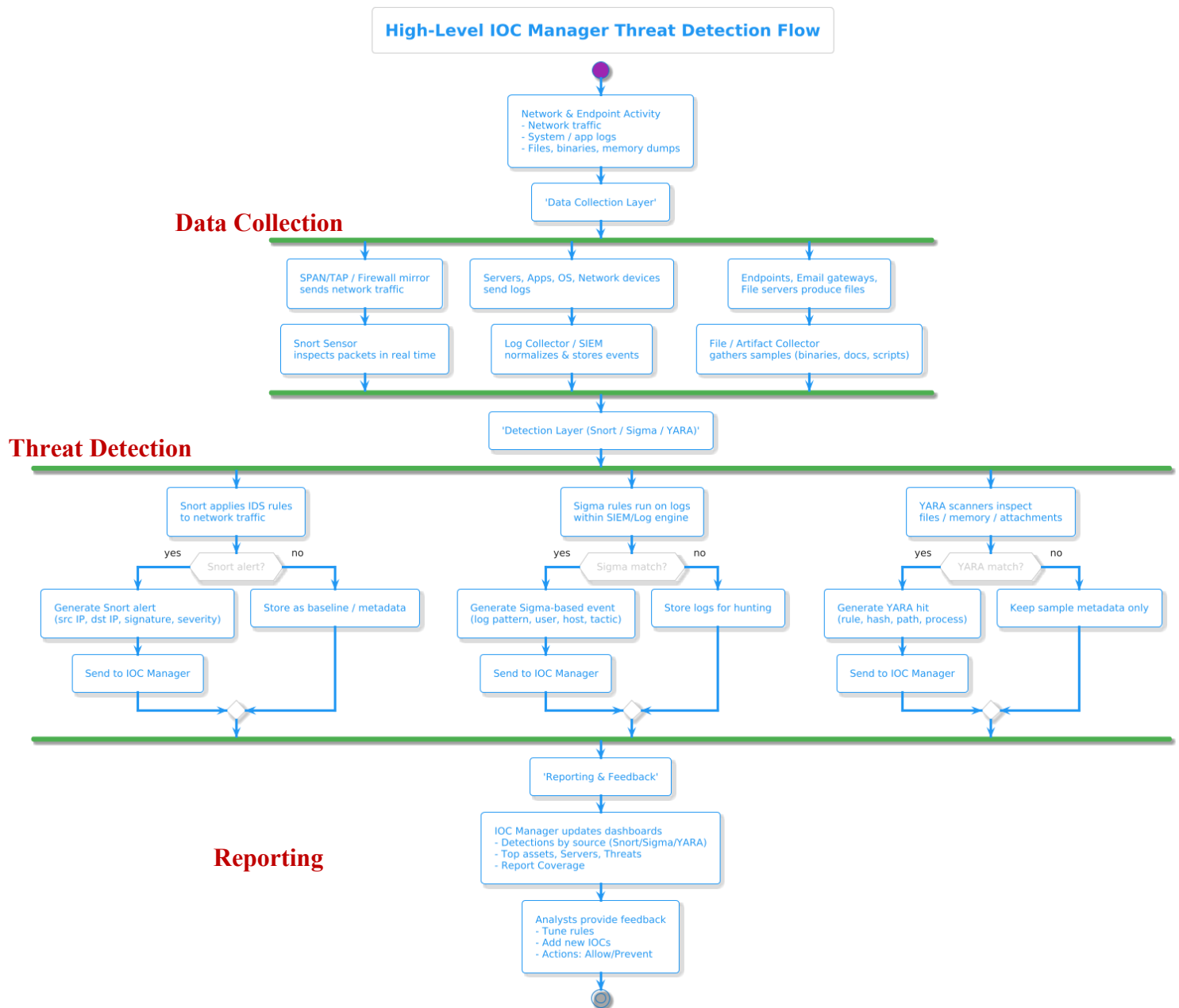


Figure 39: High-level IoC Manager threat-detection flow, illustrating how data is collected, processed and forwarded to the IoC Manager for centralized alerting, dashboards and analyst feedback.

4.3.2.3 Sequence Diagram

The following sequence diagram illustrates how users initially interact with the IoC Manager through the login and authentication process.

- Login and Authentication Diagram

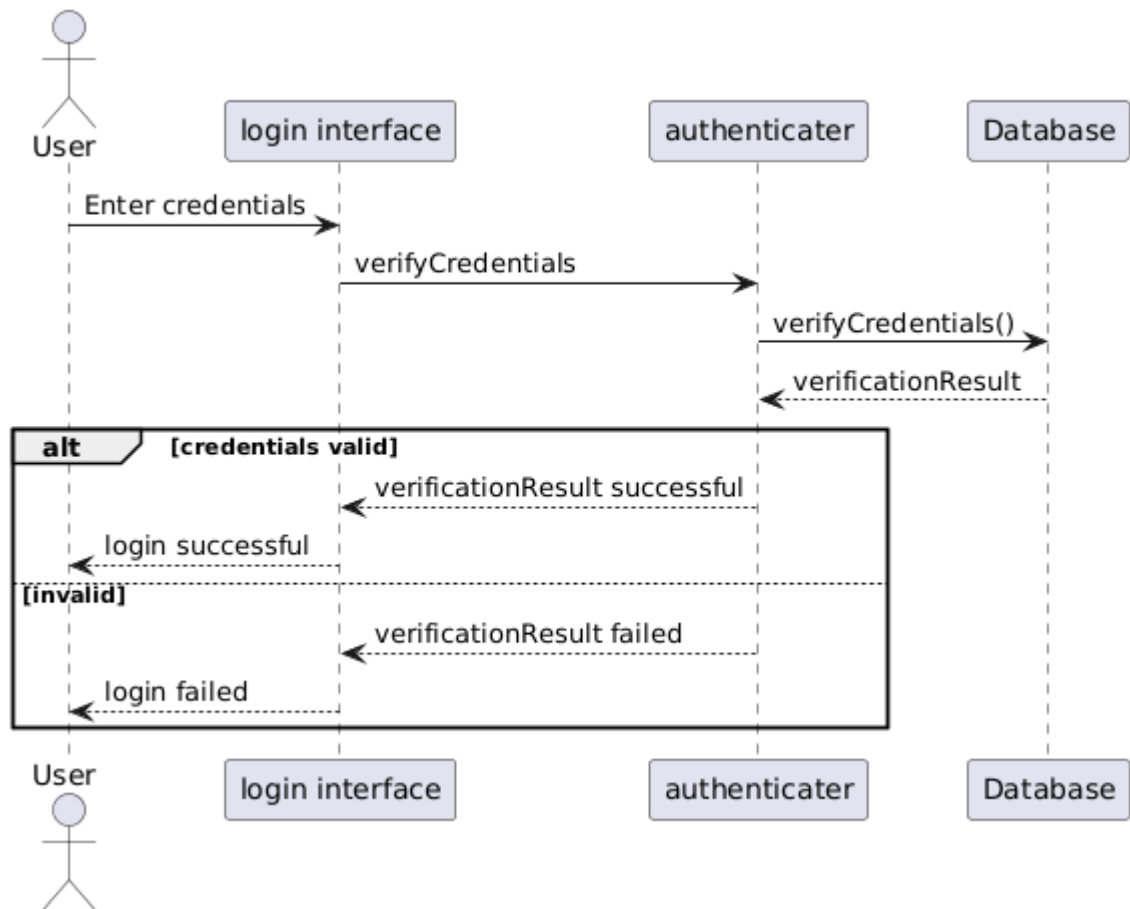


Figure 40: A sequence diagram illustrates how users initially interact with the IoC Manager through the login and authentication process.

- Admin Role Sequence Diagram

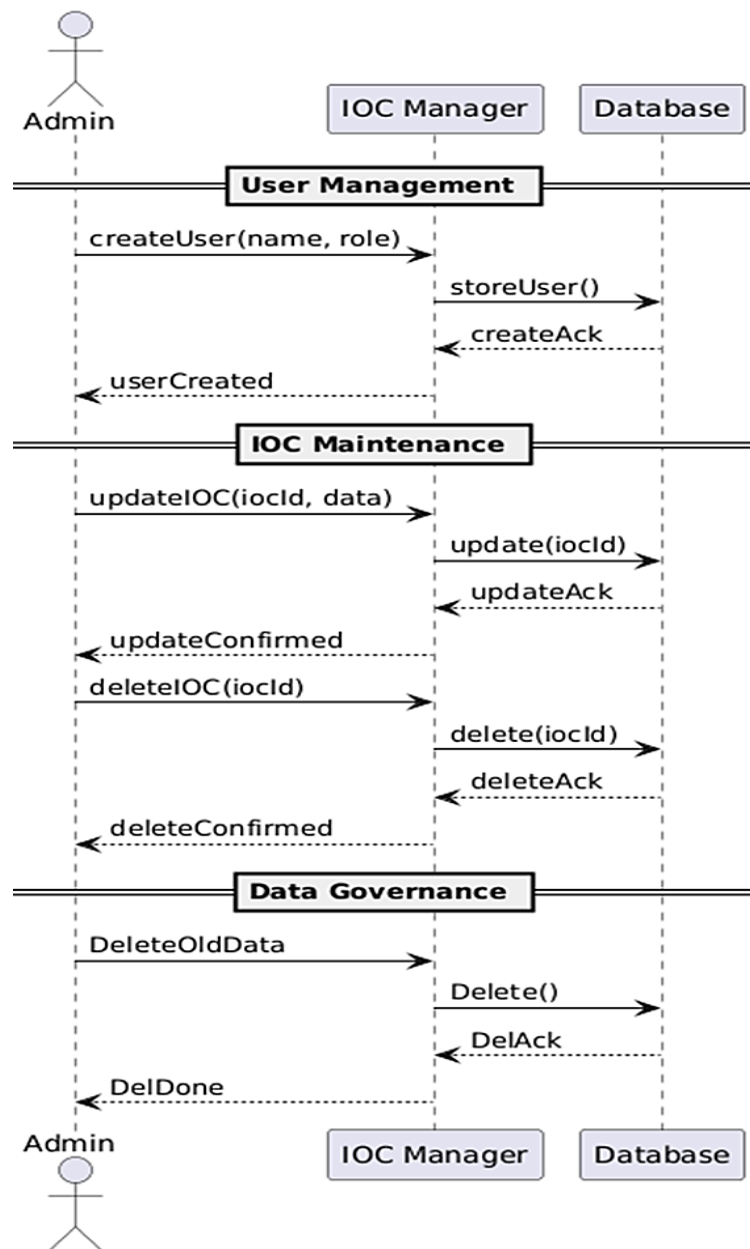


Figure 41: A sequence diagram shows the main interactions carried out by the Admin role within the IoC Manager. It highlights administrative tasks such as creating new user accounts, modifying existing IOC records, and performing data governance operations.

- IT Role Sequence Diagram

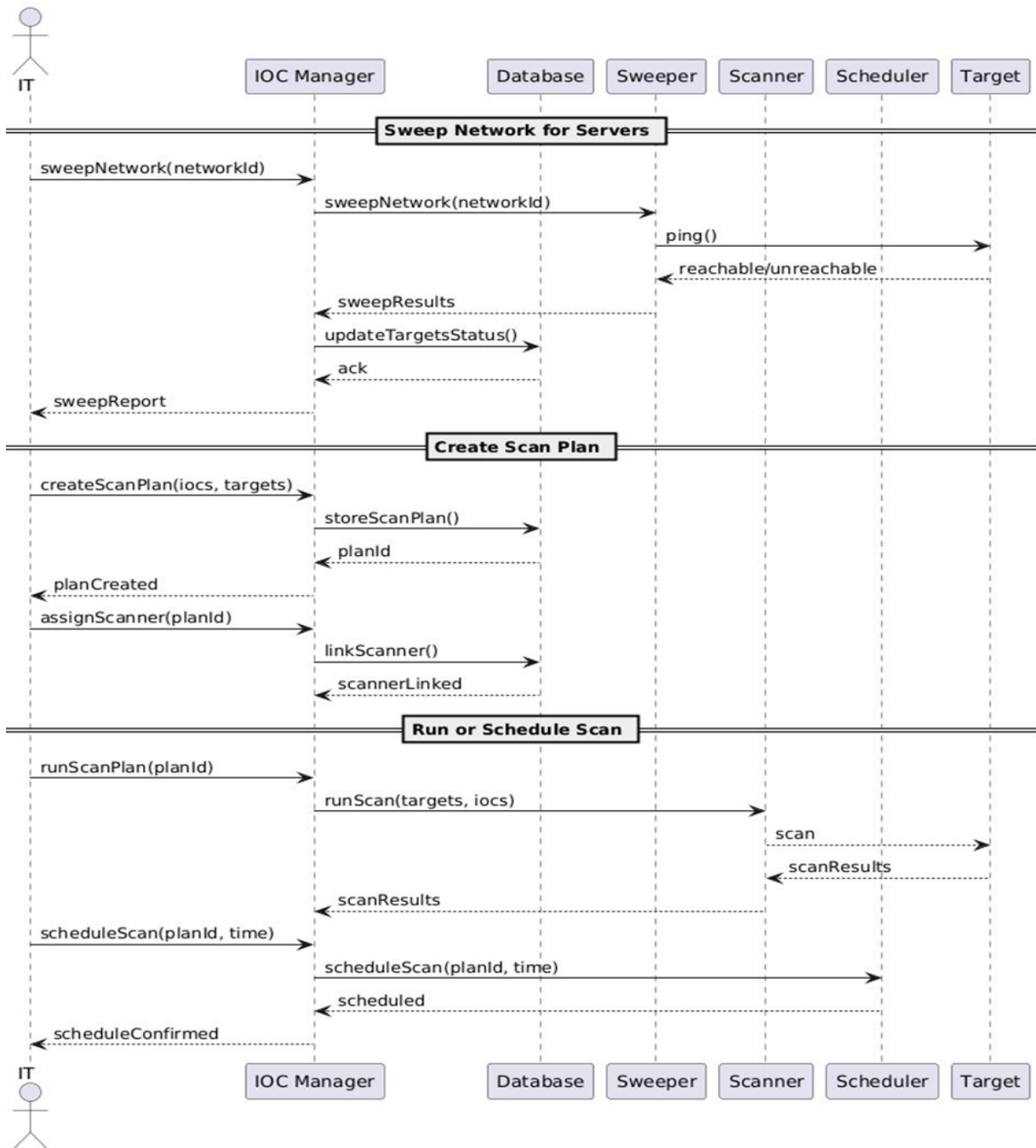


Figure 42: A sequence Diagram that focuses on the workflow of the IT role, which is responsible for setting up operational environments within the IoC Manager.

- Analyst Role Sequence Diagram

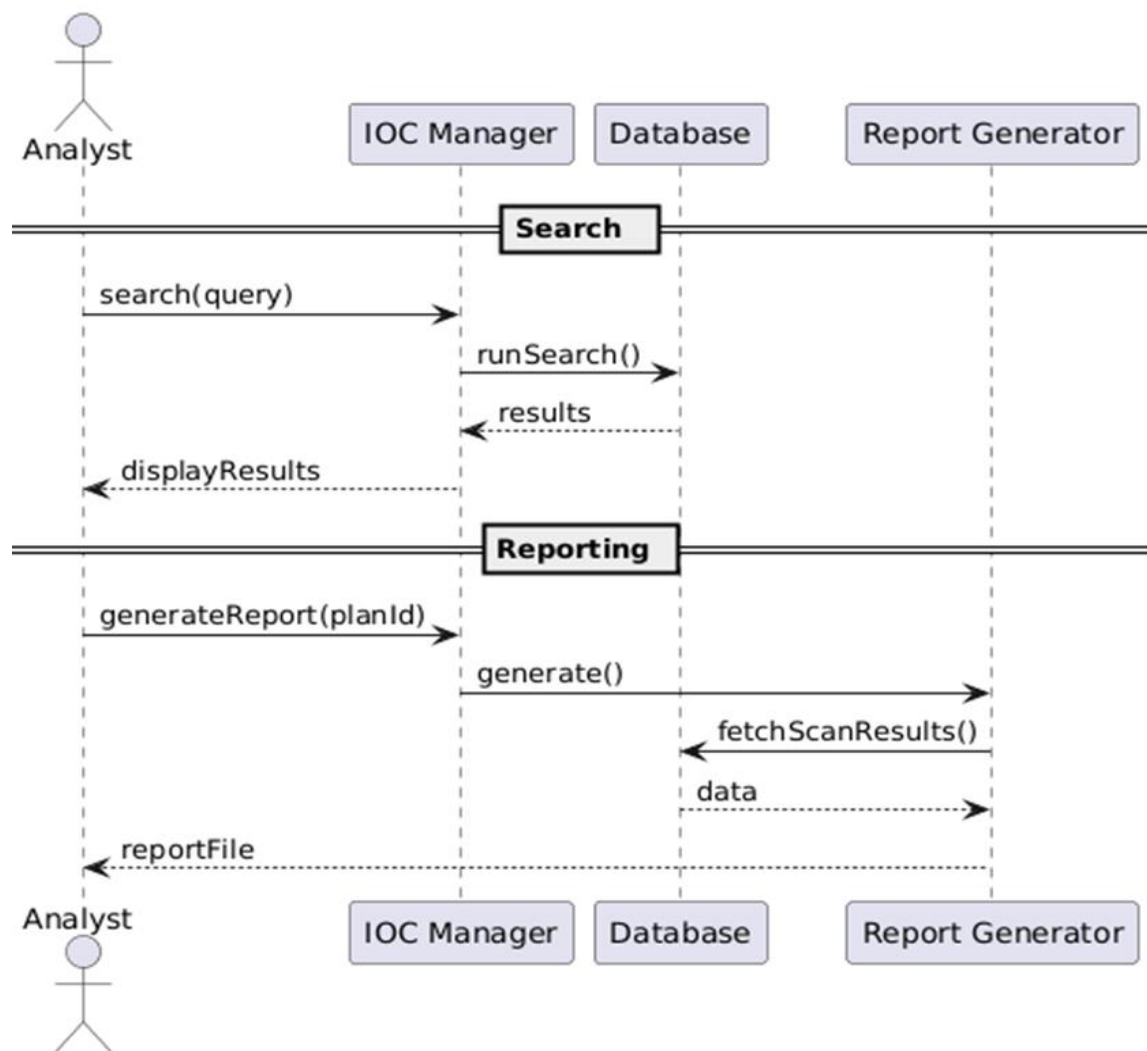


Figure 43: A sequence diagram that outlines the interaction flow for the Analyst role, who primarily works with searches, and reports.

4.4 Data Modeling

This section presents the **data model** of the IoC Manager system. It describes how project information is structured in the database by identifying the main entities (such as Sweeper, Network, Target, ScanJob, ScanResult, IOC, Alert, and Report), their attributes, and the relationships between them.

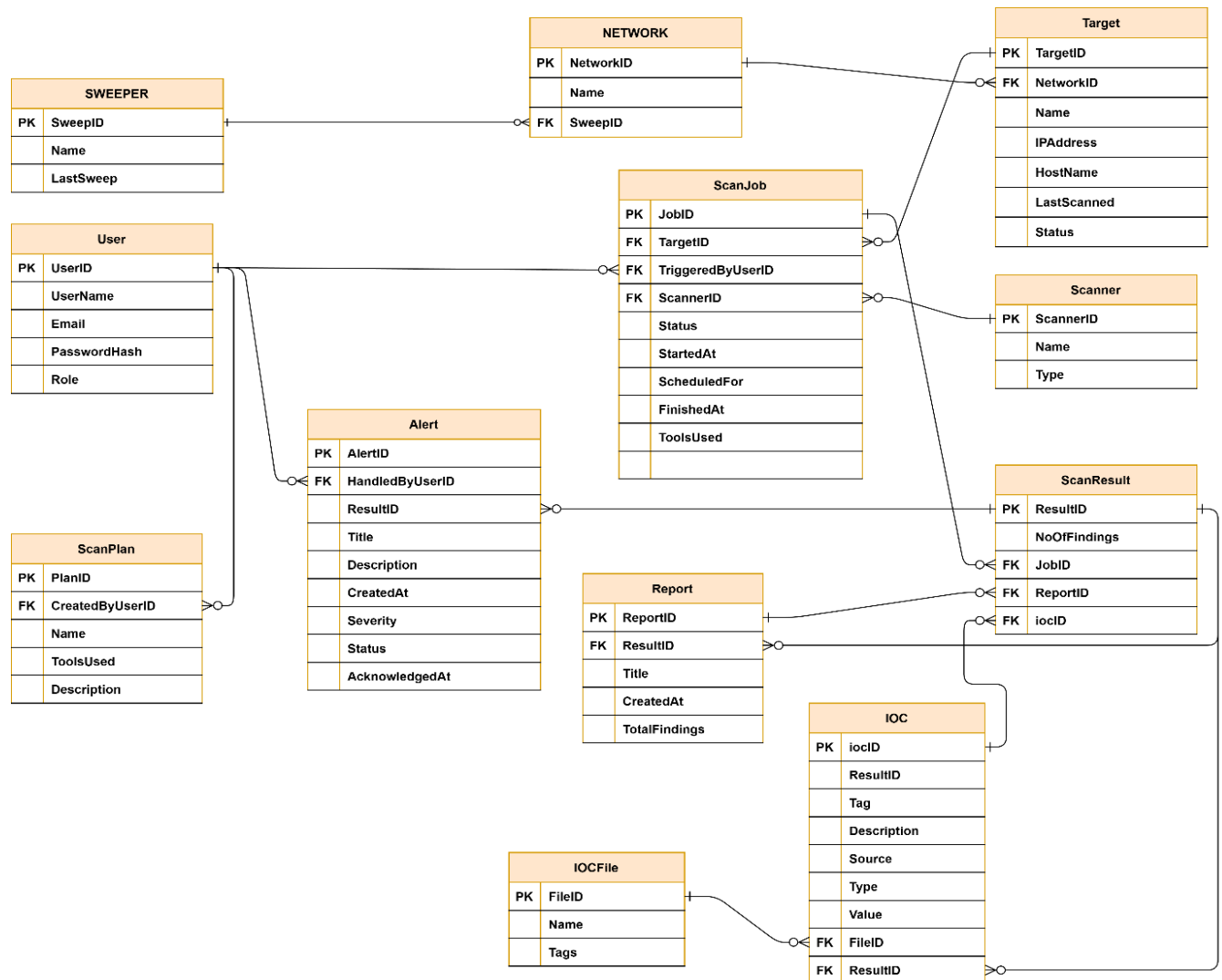


Figure 44: Relational ER Diagram (Logical Data Model) showing the main tables with their attributes, primary keys and foreign-key relationships.

4.5 User Interface Design

This section presents the main dashboard design of the IoC Manager. It outlines the layout and navigation of key screens (such as main dashboard, server discovery, rule management, and reports) to show how Admin, Analyst, and IT users will interact with the system and access its core functions.



Figure 45: a dashboard providing an overview.

IoC Manager

Dashboard

MANAGEMENT

Activity

Snort Rules

Sigma Rules

YARA Rules

OPERATIONS

Threat Feeds

Servers

Distribution

Reports

ANALYSIS

Analytics

ADMINISTRATION

Audit Logs

Settings

Activity

Home / Activity

Search...

ImportAdd IoC

ID	Type	Value	Risk	Confidence	Status	Tags	Occurrences	Actions
1	IP	192.168.1.100	Critical	95	Active	malware, c2	1	  
2	Domain	malware.example.com	High	88	Active	phishing	2	  
3	Hash	a1b2c3d4e5f6789	High	92	Blocked	trojan	1	  
4	URL	http://phish.net/login	Critical	91	Active	phishing	5	  
5	Email	attacker@evil.com	Medium	75	Active	spam	1	  
6	IP	1.1.1.1	Low	30	Active	dns	1	  

Figure 46: a specific overview of IoC activity.

IoC Manager

Dashboard

MANAGEMENT

Activity

Snort Rules

Sigma Rules

YARA Rules

OPERATIONS

Threat Feeds

Servers

Distribution

Reports

ANALYSIS

Analytics

ADMINISTRATION

Audit Logs

Settings

Activity

Home / Activity

Search...

ID	Type	Value	Risk	Confidence	Status	Tags	Occurrences	Actions
1	IP	192.168.1.100	Critical	95	Active	malware, c2	1	  
2	Domain	malware.example.com	High	88	Active	phishing	2	  
3	Hash	a1b2c3d4e5f6789	High	92	Blocked	trojan	1	  
4	URL	http://phish.net/login	Critical	91	Active	phishing	5	  
5	Email	attacker@evil.com	Medium	75	Active	spam	1	  
6	IP	1.1.1.1	Low	30	Active	dns	1	  

Database Entry

IOC ID:

1

IOC:

192.168.1.100

IOC Type:

IP

Threat Type:

Critical

Confidence Score:

95

Critical

Status:

Blocked

Tags:

malware c2

Reporter:

socanalyst (Internal)

First Seen:

2024-11-15 10:30

Notes:

C2 server

Close

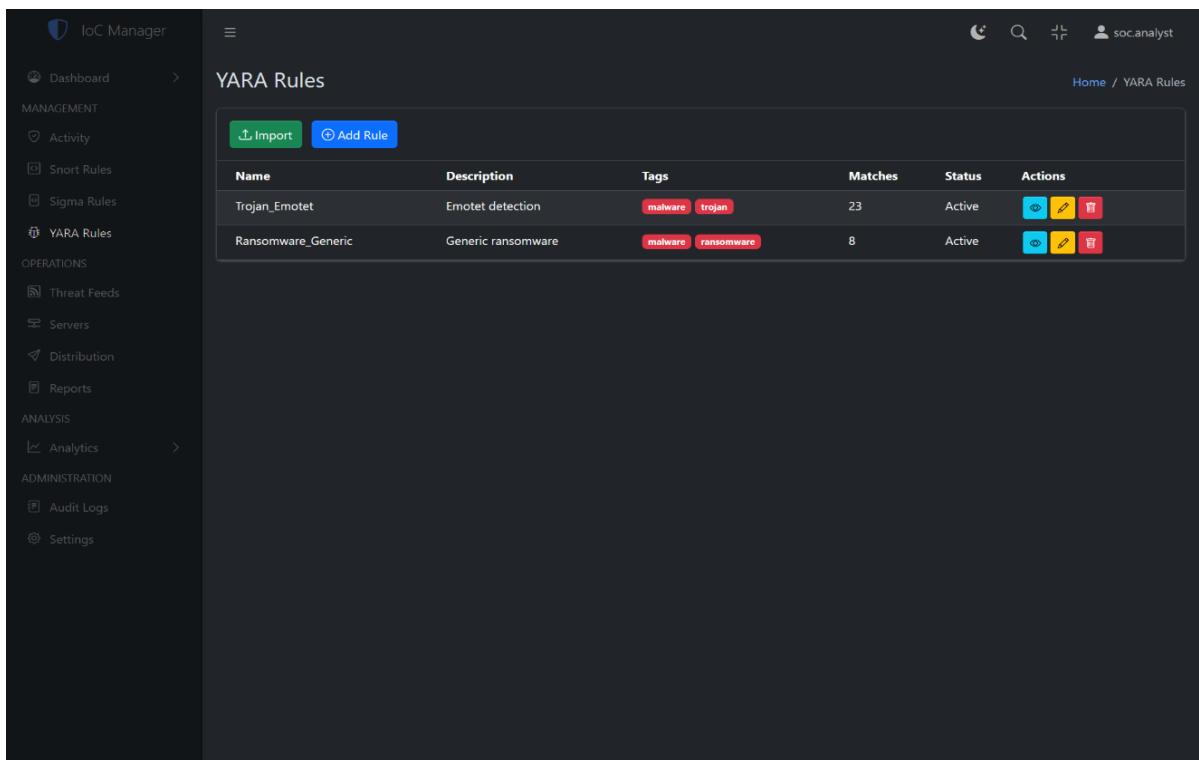


Figure 47: 48 subpage detailing YARA rules. You can import rules or add one specific rule.

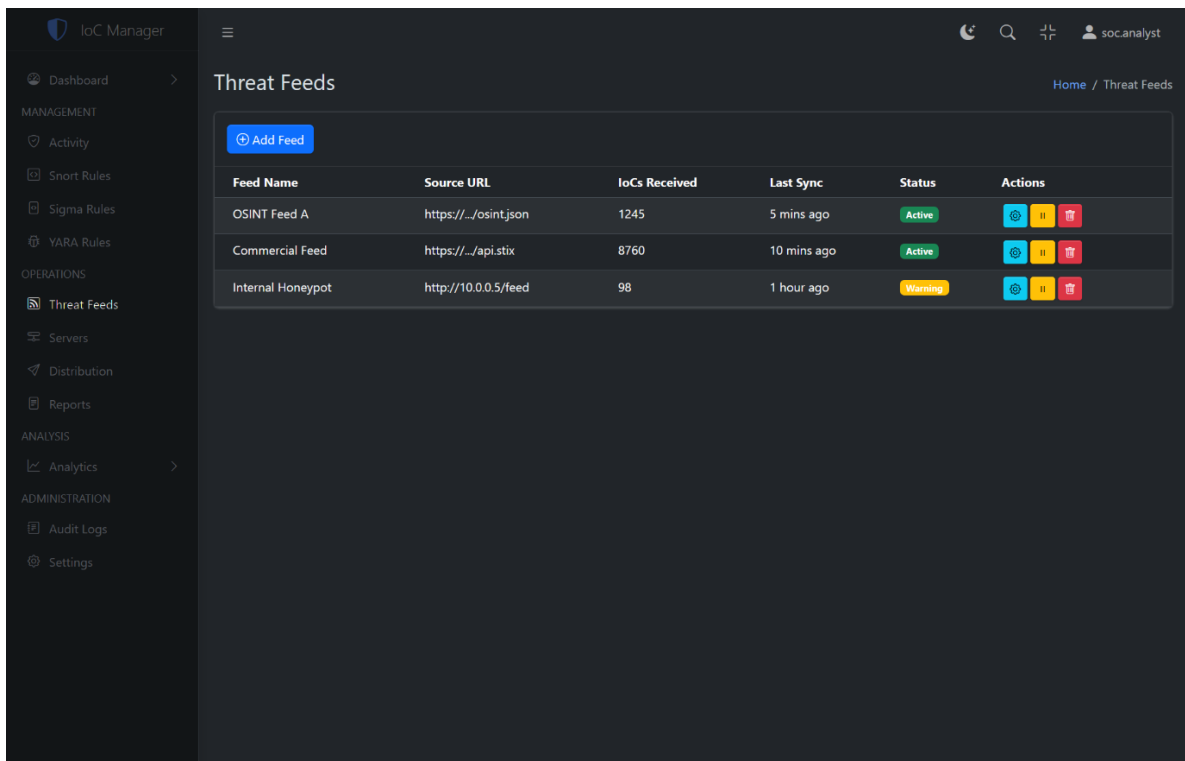


Figure 49: an overview of Threat Feeds, while being able to add more.

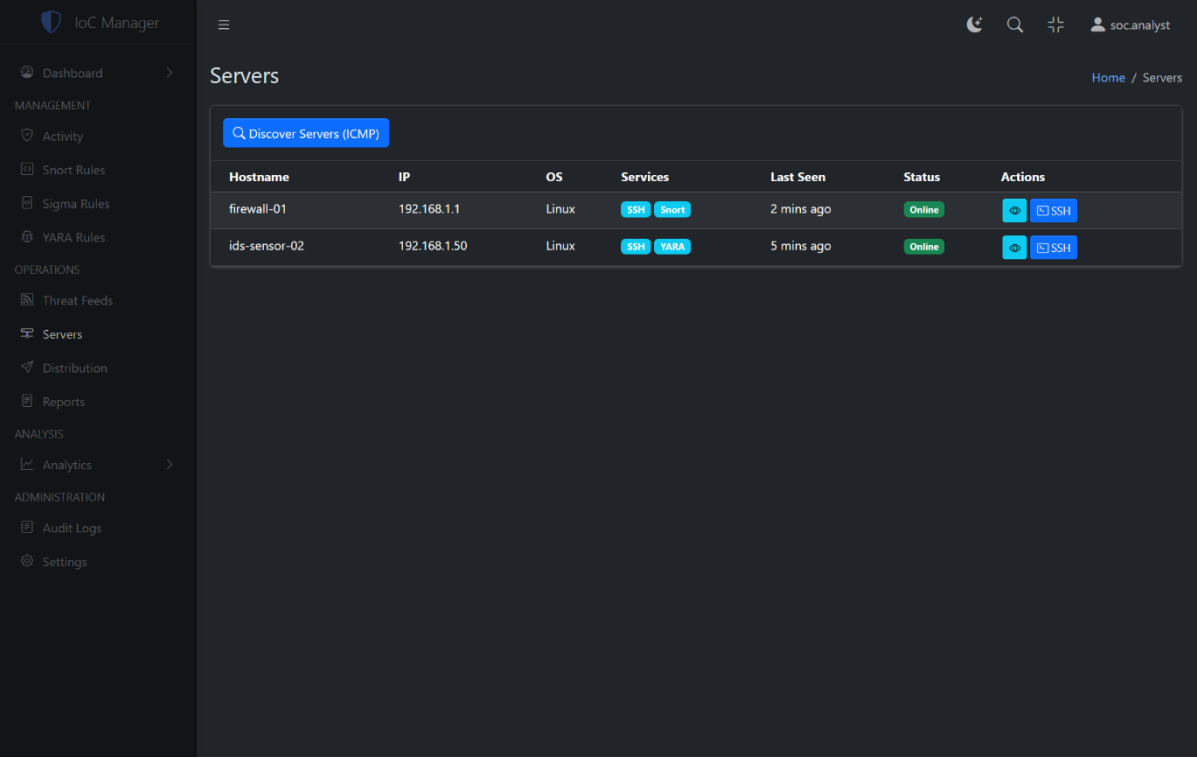


Figure 50: Servers subpage which includes current servers and ability to connect to them. You can discover more servers.

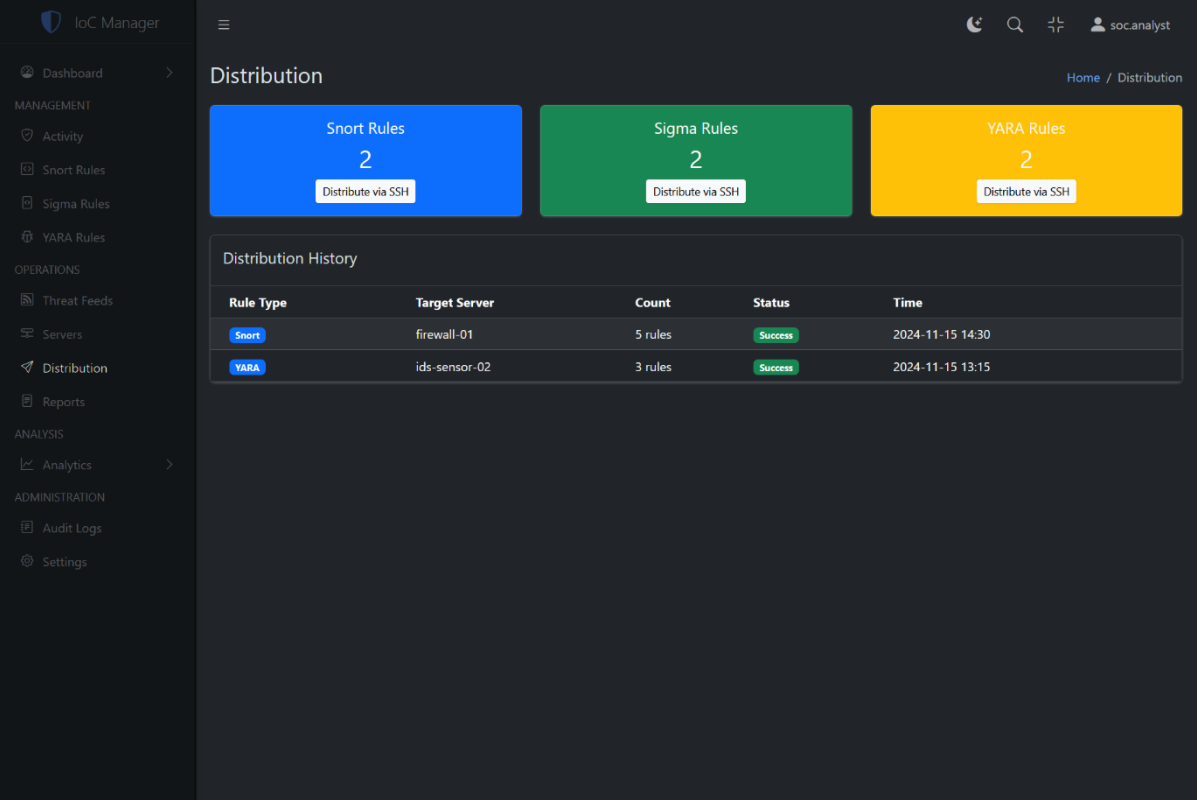


Figure 51: Distribution subpage.

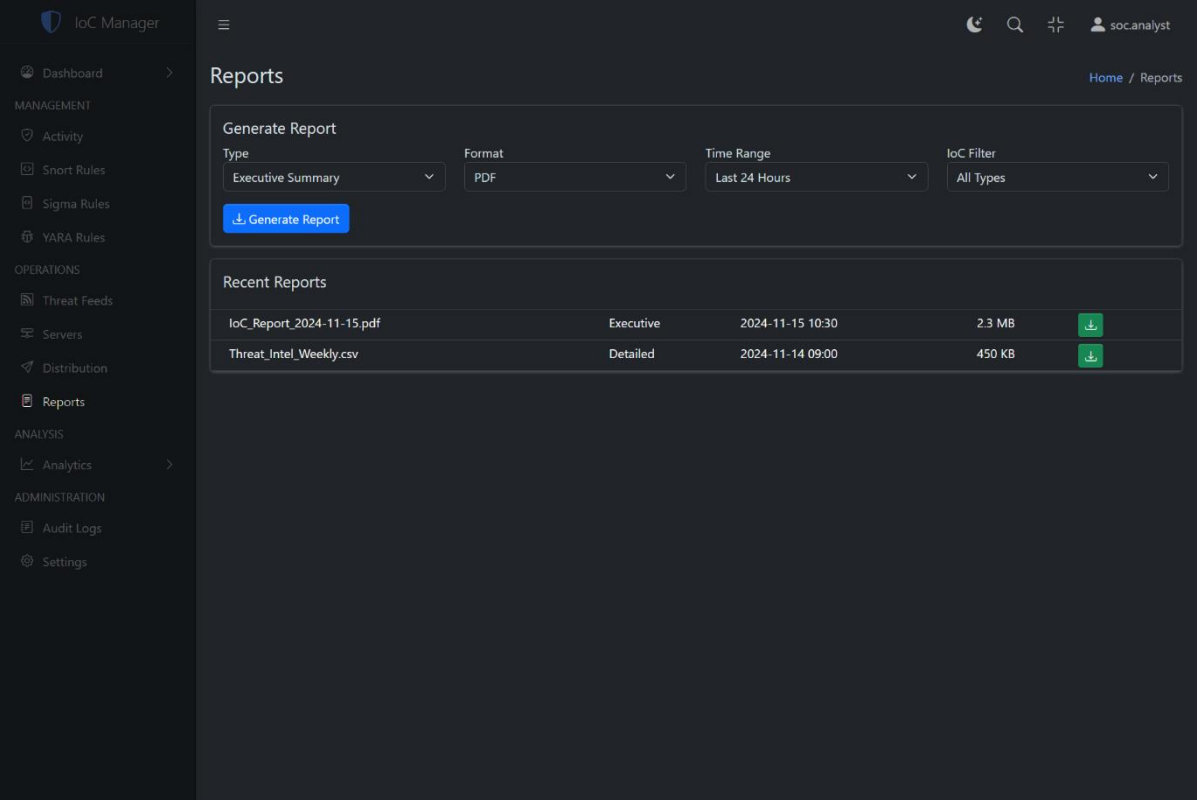


Figure 52: you can generate specific types of reports, with options for formats, time and filters.



Figure 53: a visualization of data.

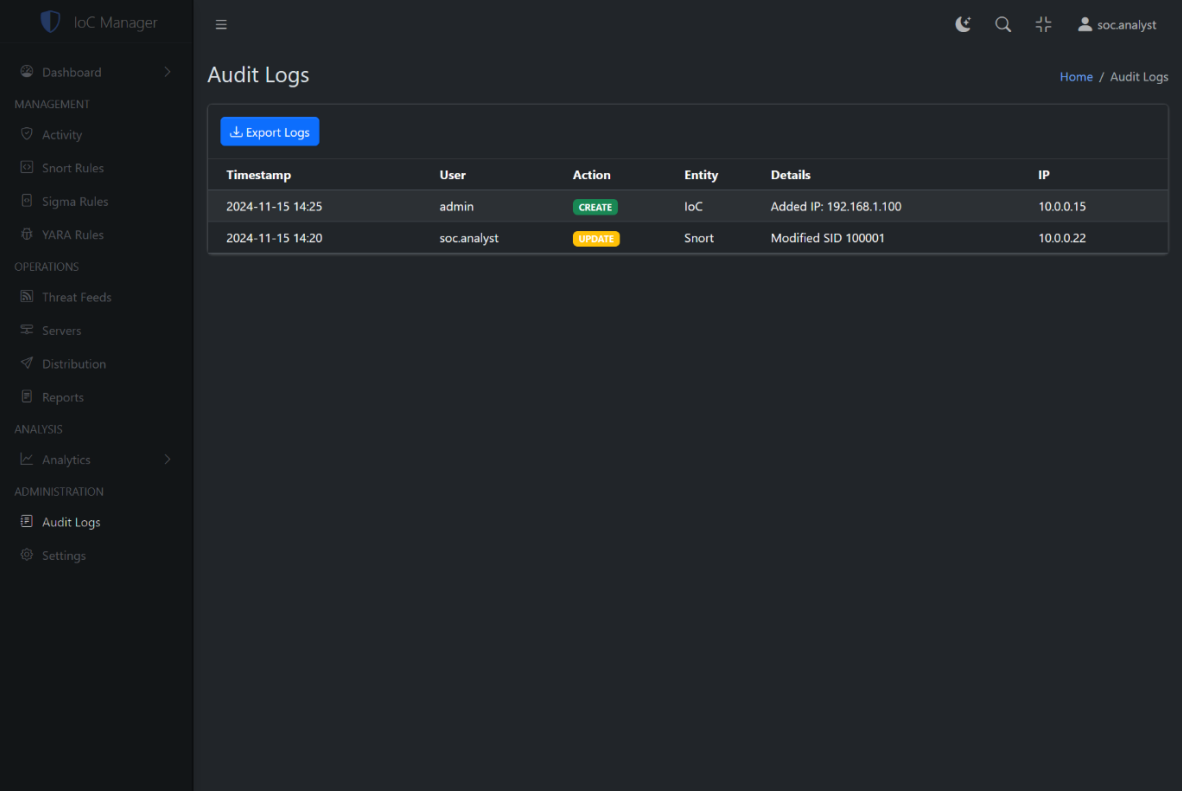


Figure 54: history of actions for who did what.

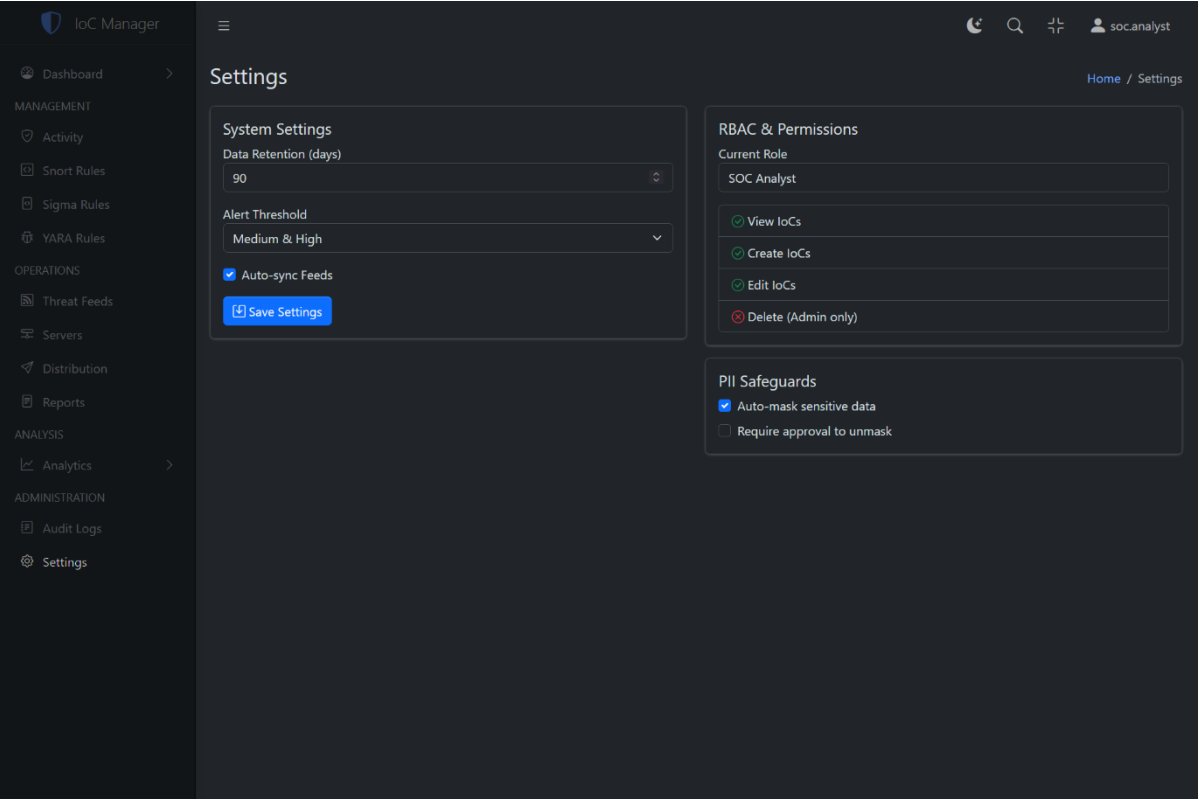


Figure 55: user settings.

4.6 Proposed Solution

The proposed IoC Manager uses a set of deterministic backend workflows and algorithms to support server discovery, rule distribution, data ingestion, and alerting. The main techniques are summarized below.

4.6.1 Server Discovery Algorithm

The server discovery module implements an ICMP sweep over a user-defined IP range. For each address in the range, the system validates the network, sends an ICMP echo request, waits for a reply within a configurable timeout, and records the host as “up” or “down”. The results are stored in the database and exposed to the UI as the list of discovered servers that can later receive Snort/YARA/Sigma rules.

4.6.2 Rule Distribution Workflow

The rule-distribution module takes a selected Snort, YARA, or Sigma rule file and a set of target servers, creates an export job, and iterates over the servers. For each server, it opens an SSH session using the stored connection parameters, uploads the rule file to the appropriate directory and updates the job status (success/failure) in the database. Failed transfers are logged and alerted so that the user can re-run the job or fix connectivity issues.

4.6.3 Ingestion and Normalization Pipeline

The ingestion pipeline accepts Snort alerts and rule files as input. Snort alerts are parsed to extract fields such as timestamp, source IP, destination IP, ports, protocol, signature message, and rule ID, which are then mapped to the IoC schema before being stored. Sigma and YARA rules are stored together with their metadata (ID, title, tags, author). This normalization step allows the dashboard and search functions to work over a single, consistent data model regardless of the originating scanner.

4.6.4 Alerting and Dashboard Logic

The alerting logic evaluates incoming detections against simple conditions (for example, severity, IoC status, or server) and flags high-priority events as alerts. These alerts are pushed to the dashboard, where they are highlighted for the user until they are acknowledged. Summary queries over the normalized data are used to populate charts such as “top attacker IPs” and “IoCs by Geolocation”.

4.6.5 IoC Confidence Scoring (*Future Enhancement*)

If enabled, the system can assign a basic score to each IoC using factors such as type, source, age, status, etc. The score is stored as an attribute and used for sorting and visual prioritization in the UI to focus the security team effort on the most critical IoCs.

4.7 Summary

In this chapter, the complete system design of the IoC Manager was presented. The chapter began in **Section 4.2** by outlining the architectural design, which described how the system components were organized and how they interacted with distributed servers running Snort, YARA, and Sigma. The three-layer architecture provided a structured approach that clearly separated the interface, logic, and data layers.

The object-oriented design was detailed in **Section 4.3**, where structural and behavioral models were used to explain how the system operated internally. Class diagrams, activity diagrams, state machine diagrams, and sequence diagrams illustrated the workflows previously defined in **Chapter 3**, such as server discovery (related to **3.2**), rule distribution (supporting requirements from **3.3.1**), and scan execution (linked to use cases in **3.4**).

In **Section 4.4**, the data modeling view showed a relational ER diagram that defines the main database entities of the IoC Manager, their key attributes, and the primary–foreign key relationships between them. This logical data model captures how sweep results, scan jobs, IoC files, detections, alerts, and reports are stored and linked together so that the system can reliably query, correlate, and present threat information.

The user interface design in **Section 4.5** then demonstrated how different user roles—Admin, IT, and Analyst—interacted with the system. These screenshots reflected the use cases specified earlier in **3.4**, such as login, scanning, server management, and report generation.

Finally, **Section 4.6** summarized the proposed solution by describing the main algorithms and workflows used in the IoC Manager, including ICMP-based server discovery, SSH-based rule distribution, the ingestion pipeline, alerting logic, and planned enhancements.

Overall, this chapter transformed the analysis from **Chapter 3** into a complete design blueprint for the IoC Manager, forming the basis for the implementation work that would follow in Chapter 5.

5 Chapter 5: System Implementation

The students of CS/IS491 should remove this chapter and prepare it only when they take CS/IS492 in the second semester.

5.1 Introduction

5.2 Tools and Languages

In this section you should state the tools and/or languages that you intend to use and its justifications.

5.3 Mapping Design to Implementation

Detail explanation is required on how the discussion on system design (in Chapter 4) relates with the actual implementation.

5.4 Main/Most Important Codes

The student should include (at least three) selected most important codes with the explanation. Please exclude the code on interfaces, connection to the database, etc.

5.4.1 Model Building and Training

This section should include the dataset collection, dataset preprocessing, and feature extraction details. Then, the model preprocessing and training should be presented.

5.4.2 Model Testing (Experimental results)

After the training phase, the students should report the results of the model performance in terms of accuracy, F-score and other suitable metrics depending on the nature of the addressed problem.

5.4.3 System Development

This section should include the details of developing and implementing the system which detailed the implementation of the core component of the system (backend and frontend components) and the integration of the developed model into your system.

5.5 System Testing

The system testing can cover one or more of the following test types:

- Unit tests.
- Verify that all system components are integrated properly, and that actual processing situation will be handled correctly (integration test).
- System and release testing.
- Demonstrate that users can interact with the system successfully (usability tests).

5.6 Results and Discussion

The student presents his results and interprets them by making links between the required solution and the results obtained. The student writes arguments and explanations, justifications and logical deductions so that the reader is led to understand his position. All this interpretation approach must be based on facts (existing data and results) and on technical and scientific knowledge of the student.

5.7 Summary

6 Chapter 6: Conclusion and Future Work

6.1 Conclusion

In the conclusion the student should make a brief review of the problem, objectives, assumptions, and the methodology used, and summarize the solutions and the obtained results.

6.2 Goals Achieved

This section describes to which degree the reached outcome supports the original objectives of the project (partially, fully achieved, or exceeded expectations). Here you will summarize the achievements and deficiencies of your project. You may also state what you would/could have done if you had had more time or if things had worked out differently.

6.3 Limitations and Future Work

In this section the student should set out the limits of his work and suggest new works to be undertaken to complement or improve the present work.

7 References

- [1] Neo23x0 (Florian Roth), "LOKI — Simple IOC and YARA Scanner," [Online]. Available: <https://github.com/Neo23x0/Loki>. [Accessed 1 December 2025].
- [2] Start, Critical, "Pyramid of Pain," [Online]. Available: <https://www.criticalstart.com/>. [Accessed December 2025].
- [3] ICANN, "WHOIS Lookup," ICANN, [Online]. Available: <https://lookup.icann.org/>. [Accessed November 2025].
- [4] Liquid Web, "What Is GeoIP?," Liquid Web, [Online]. Available: <https://www.liquidweb.com/blog/what-is-geoip/>. [Accessed November 2025].
- [5] IMD World Competitiveness Center, "World Competitiveness Ranking / IMD World Competitiveness Yearbook," IMD, [Online]. Available: <https://www.imd.org/wcc/world-competitiveness-center/rankings/>. [Accessed November 2025].
- [6] International Telecommunication Union (ITU), "Global Cybersecurity Index (GCI) 2024," ITU, 2024. [Online]. Available: <https://www.itu.int/epublications/publication/global-cybersecurity-index-2024>. [Accessed December 2025].
- [7] Wikipedia, "YARA Wikipedia," [Online]. Available: <https://en.wikipedia.org/wiki/YARA>. [Accessed October 2025].
- [8] VirusTotal, "What is YARA?," VirusTotal, [Online]. Available: <https://docs.virustotal.com/docs/what-is-yara>. [Accessed October 2025].
- [9] YARA Project, "YARA Documentation," [Online]. Available: <https://yara.readthedocs.io/en/latest/>. [Accessed October 2025].

- [10] Huntress, "What Are YARA Rules?," Huntress, [Online]. Available: <https://www.huntress.com/cybersecurity-101/topic/what-are-yara-rules>. [Accessed October 2025].
- [11] OnlineHashCrack, "YARA Rules Writing: Detect Malware Patterns," OnlineHashCrack, [Online]. Available: <https://www.onlinehashcrack.com/guides/security-tools/yara-rules-writing-detect-malware-patterns.php>. [Accessed October 2025].
- [12] HarfangLab, "Signatures Engine: YARA Rules," HarfangLab, [Online]. Available: <https://harfanglab.io/edr/signatures-engine-yara/>. [Accessed October 2025].
- [13] S. Bavarva, "Hunting Evil with — YARA," Medium, [Online]. Available: <https://snehbavarva.medium.com/hunting-evil-with-yara-07dcd83f2781>. [Accessed October 2025].
- [14] GeeksforGeeks, "Threat Hunting Using YARA," GeeksforGeeks, [Online]. Available: <https://www.geeksforgeeks.org/ethical-hacking/threat-hunting-using-yara/>. [Accessed October 2025].
- [15] RangeForce, "Malware Detection Using YARA," RangeForce, [Online]. Available: <https://materials.rangeforce.com/tutorial/2020/02/19/Malware-Detection-using-YARA/>. [Accessed December 2025].
- [16] SOC Prime, "Sigma Rules — The Beginner's Guide," SOC Prime Blog, [Online]. Available: <https://socprime.com/blog/sigma-rules-the-beginners-guide/>. [Accessed December 2025].
- [17] SigmaHQ, "Sigma Documentation," SigmaHQ, [Online]. Available: <https://sigmahq.io/docs>. [Accessed October 2025].
- [18] SigmaHQ, "Sigma Rule Repository," GitHub, [Online]. Available: <https://github.com/SigmaHQ/sigma>. [Accessed October 2025].

- [19] Picus Security, "What Is a Sigma Rule?," Picus Security, [Online]. Available: <https://www.picussecurity.com/resource/glossary/what-is-sigma-rule>. [Accessed October 2025].
- [20] Bitdefender, "Sigma Rules," Bitdefender Business Support, [Online]. Available: <https://www.bitdefender.com/business/support/en/77209-1424444-sigma-rules.html>. [Accessed December 2025].
- [21] P. Krill, "The Greatest Open Source Software of All Time," InfoWorld, [Online]. Available: <https://www.infoworld.com/article/2278109/the-greatest-open-source-software-of-all-time.html>. [Accessed October 2025].
- [22] Help Net Security, "The Story of Snort: Past, Present, and Future," Help Net Security, [Online]. Available: <https://www.helpnetsecurity.com/2005/10/24/the-story-of-snort-past-present-and-future/>. [Accessed October 2025].
- [23] A. Ahmed and M. N. Islam, "An Implementation of Intrusion Detection System Using Snort," [Online]. Available: <https://arxiv.org/pdf/1204.1336>. [Accessed October 2025].
- [24] Wikipedia, "Snort (software)," Wikipedia, The Free Encyclopedia, [Online]. Available: [https://en.wikipedia.org/wiki/Snort_\(software\)](https://en.wikipedia.org/wiki/Snort_(software)). [Accessed October 2025].
- [25] Snort Project, "Snort Users Manual 2.9," Snort Project, [Online]. Available: <https://www.snort.org/documents/snort-users-manual>. [Accessed October 2025].
- [26] Zenarmor, "What is Snort?," [Online]. Available: <https://www.zenarmor.com/docs/network-security-tutorials/what-is-snort>. [Accessed October 2025].
- [27] S. Kumar and R. Patel, "Network Intrusion Detection and Prevention Systems Using Snort Approach," Academia.edu, [Online]. Available: https://www.academia.edu/33927051/NETWORK_INTRUSION_DETECTION_AND_PREVENTION_SYSTEMS_USING_SNORT_APPROACH. [Accessed October 2025].

- [28] N. Khamphakdee, N. Benjamas and S. Saiyod, "Example of Snort IDS Rule," ResearchGate, [Online]. Available: https://www.researchgate.net/figure/Example-of-Snort-IDS-Rule-The-rule-options-of-Snort-consist-of-two-parts-a-keyword-and_fig1_281564631. [Accessed October 2025].
- [29] MISP Project, "MISP – Open Source Threat Intelligence Platform," MISP Project, [Online]. Available: <https://www.misp-project.org/>. [Accessed november 2025].
- [30] Kaspersky, "MISP Integration Guide," Kaspersky Threat Intelligence Portal, [Online]. Available: <https://support.kaspersky.com/help/TIDFMISP/2.2/en-US/174236.htm>. [Accessed November 2025].
- [31] Filigran, "OpenCTI Platform Overview," [Online]. Available: <https://filigran.io/platforms/opencti/>. [Accessed November 2025].
- [32] OpenCTI, "Data Model Documentation," OpenCTI, [Online]. Available: <https://docs.opencti.io/latest/usage/data-model/>. [Accessed November 2025].
- [33] Fortinet, "FortiAnalyzer – Analytics and Log Management," Fortinet, [Online]. Available: <https://www.fortinet.com/products/management/fortianalyzer>. [Accessed November 2025].
- [34] Nomios, "Fortinet FortiAnalyzer," Nomios, [Online]. Available: <https://www.nomios.com/partners/fortinet/management-and-visibility/fortianalyzer/>. [Accessed November 2025].
- [35] MITRE, "CVE — Common Vulnerabilities and Exposures," MITRE Corporation, [Online]. Available: <https://cve.mitre.org/>. [Accessed November 2025].

Activity Home / Activity

Import
Add IoC

ID	Type	Value	Risk	Confidence	Status	Tags	Occurrences	Actions
1	IP	192.168.1.100	Critical	95	Blocked	malware, c2	1	
2	Domain	malware.example.com	Critical	88	Blocked	phishing	2	
3	Hash	a1b2c3d4e5f6789	Low	24	Blocked	trojan	1	
4	URL	http://phish.net/login	Critical	91	Active	phishing	5	
5	Email	attacker@evil.com	High	48	Active	spam	1	
6	IP	1.1.1.1	Medium	30	Active	dns	1	

Database Entry ×

IOC ID:	1
IOC:	192.168.1.100
IOC Type:	IP
Threat Type:	Critical
Confidence Score:	95 Critical
Status:	Blocked
Tags:	malware c2
Reporter:	soc.analyst (Internal)
First Seen:	2024-11-15 10:30
Notes:	C2 server

Close

IoC Manager

Dashboard
Activity
Short Rules
Sigma Rules
YARA Rules
Threat Feeds
Servers
Distribution
Reports
Analytics
Administration
Audit Logs

Activity

Add IoC

ID	Type	Value	Risk	Confidence	Status	Tags	Occurrences	Actions
1	IP	192.168.1.100	Critical	95	Blocked	malware, c2	1	
2	Domain	malware.example.com	Critical	88	Blocked	phishing	2	
3	Hash	a1b2c3d4e5f6789	Low	24	Blocked	trojan	1	
4	URL	http://phish.net/login	Critical	91	Active	phishing	5	
5	Email	attacker@evil.com	High	48	Active	spam	1	
6	IP	1.1.1.1	Medium	30	Active	dns	1	

Database Entry ×

Close

Check comment

The purpose of a reference is to acknowledge the contributions of other authors to which you owe an intellectual debt, and to enable readers to locate the source easily. In this section, you can use alphabetical or numerical system. For the latter we recommend IEEE system which is widely used in computer sciences and engineering. IEEE system dictates the in-text citation to appear as a number within square brackets (e.g. [1]). The full details of the reference appear in the reference list in the order of citation in the text. As for the alphabetical system (also known as author-date system), Harvard or APA system of referencing can be used. The advantage of author-date system is that the name of the author and the year appear in the text, hence this ease reading. The reader only refers to the reference section for the details of the document since the name and the year already exist in the text.

The following is an example list of references using IEEE system. Please refer to the IEEE citation reference for details.

- [1] Yara website I. Sommerville, "Software Engineering", 11th ed., Addison-Wesley, 2015.
- [2] V. J. Blue, and J. L. Adler, "Cellular automata micro-simulation of bi-directional pedestrian flows," *J. Transportation Research*, pp. 135-141, 2000.
- [3] S. Sarmady, F. Haron, and A. Z. H. Talib, "Modelling groups of pedestrians in least effort crowd movements using cellular automata," in *Proc. 2009 2nd Asia International Conference on Modelling & Simulation*, Bali, Indonesia, 2009, pp. 520-525.
- [4] F. H. Hassan, "Heuristic search methods and cellular automata modeling for layout design," Ph.D dissertation, Sch. of Info. Sys, Comp. and Math., Brunel Univ., UK, 2013.
- [5] G. K. Still. (2010, July 15). *Crowd Disasters* [Online]. Available: <http://www.gkstill.com/CrowdDisasters.html>.

8 Appendix A

For the graduation project CS492 report, you should prepare an appendix explaining file structure on the CD submitted with it. The appendix must also contain information on how the code should be run (i.e., the user guide or manual). Other appendices may include documents such as: the checklist of examiners' comments, questionnaire, selected experimental data, schedules, testing strategy or risk management plans. Do not include the source code as an appendix (submit it on a CD). Do not include voluminous appendices (these should also be submitted on a CD, if necessary).

Students who have successfully completed their graduation project, CS492 oral examination and made all the revisions and corrections required by the examining committee, must submit one bound copy to the Project Committee.

9 Appendix B

This section shows examples of figure, table and equation. We assume that the figure, table and equation appear in Chapter 2, therefore their numberings will be preceded by number 2. Whenever possible please place the figures and tables alongside with the captions, immediately following the first text that refer to it. If this is not possible, then the figure or table should appear in the following page. More than one figure or table can be placed in a single page. However, it is advisable to disperse the figures and tables throughout the report. Please ensure that the figures and tables do not run across pages.

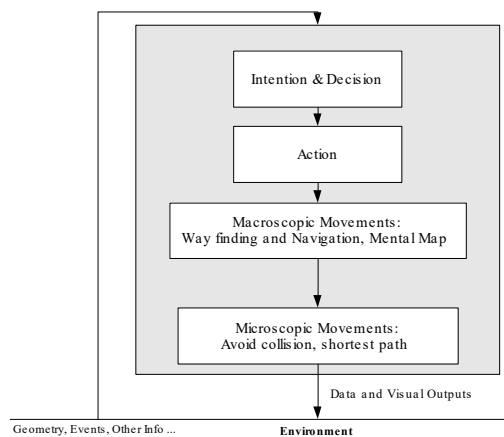


Figure 2.1: Basic movement process model.

Figure 2.1 shows that before and after spacing should be equal to 12pt. The figure caption is under the figure. Figures are centre justification. If you start a paragraph with the word “Figure” then use capital F, otherwise small f.

Tables are similar to figures, but the difference is that the table caption is above the table, for example, table 2.1 shows an example of way-finding path table concept.

Table 2.1: An example of way-finding path table concept [2].

Source	Destination	Cost	MidP-1	MidP-2	MidP-3	MidP-4	MidP-5	MidP-6
Gate1	Gate2	1	-	-	-	-	-	-
Gate1	Gate6	2	Room1	-	-	-	-	-
Room4	Room7	3	Room3	Room5	-	-	-	-
...	...	...	...	...	...	...	...	...

References are numbered in square brackets, “[” and ”]”. We use IEEE system in our citation above, see the examples in the reference section.

Each equation in a chapter is to be numbered consecutively using a decimal system appearing flush with the right-hand margin. For example:

$$Y = mx + b \qquad (2.1)$$

The numbers in parentheses are the chapter number and equation number respectively. Every new symbol used in the report text for the first time must be explained. When many special symbols are used, it is permissible to collect them in a table or in a special appendix.

1 Appendix C

The following describe the details of the required report format.

Paper

Standard A4 size; Weight: 90 Grams
Width: 8.27"; Height: 11.69"

Fonts, Type Styles

Font Size = 12 (Normal Text)

Font = Times New Roman

Heading 1 (Font Size) = 22 (Bold), Font = Times New Roman

Heading 2 (Font Size) = 16 (Bold), Font = Times New Roman

Heading 3 (Font Size) = 14 (Bold), Font = Times New Roman

Margins

Top = 1.0" Bottom = 1.0"

Left = 1.25" Right = 1.0"

Spacing

Line Spacing = 1.5

Paragraph Spacing = 6 pts (before) and 0 pts (after)

Indentation

Indent all quotations comprising 4 or more lines by 5 spaces from left.

Page Numbers

Except for the title page, number all pages which come before the first page of the body chapters consecutively with lower case roman numerals (i, ii, iii, iv...).

The first page with Arabic numeral (1, 2, 3, and so on) starts from the page of the introduction but it is mentioned on page 2 onwards. Mention page numbers on the bottom right of the page.

The first page of each section or chapter will not carry the page number, however the page number will be counted for the proceeding page.

Headers

The header will comprise the title of the project report (together with the project logo if you wish). On every odd page will appear the title of the report while on the even pages the title of the chapter or section will be mentioned. The first page of every section or chapter shall not carry the header.